

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

4. STM programozási környezet: CubeIDE, ST-Link

Írta: Hadarics Szabolcs

Lektorálta: Girgász Péter Ákos

Az előző tananyagrészben megismerkedhettünk a hardverközei programozás alapjaival és folyamatával. Egy adott program megírására nagyon sok lehetőség van, kezdve az egyszerű jegyzetomb alkalmazástól a manapság elterjedt fejlesztőkörnyezetekig. Ezek teszik lehetővé a mérnökök számára, hogy a programozás folyamatának nagy részét (kód szerkesztés, fordítás, linkelés) egy programcsomag vegye át, és ne kelljen azokkal egyesével bajlódni. Ebben a részben az STMicroelectronics 32 bites mikrokontrollereihez és mikroprocesszoraihoz készült fejlesztőkörnyezetről (az STM8 sorozathoz más fejlesztőkörnyezet tartozik), az STM32CubeIDE-ről, és a hozzá tartozó kiegészítő programok használatáról lesz szó.

Kitekintés:

Fontos itt megemlíteni, hogy a tananyag írásakor az STM32CubeIDE legfrissebb verziója az 1.5.1-es kiadás volt. Az itt bemutatott felületi elemek és működés a későbbi frissítések hatására megváltozhat, ennek ellenére javasoljuk mindig a legfrissebb verzió használatát.

STM32CUBEIDE

TELEPÍTÉS

Ahhoz, hogy a fejlesztői környezetet használni tudjuk, először fel kell telepítenünk a számítógépre. A szoftver ingyenesen elérhető bárki számára egy rövid regisztrációt követően az STMicroelectronics honlapjáról (<https://www.st.com/en/development-tools/stm32cubeide.html>). Az oldalon lejjebb görgetve az alábbi táblázatból válasszuk ki az operációs rendszerünknek megfelelő telepítőt, a verziónál pedig érdemes mindig a legfrissebbet választani. A **Get Software** gombra kattintva el kell fogadni a felhasználási feltételeket, illetve név és email cím megadását követően a megadott címre kapjuk meg a letöltéshez vezető linket.

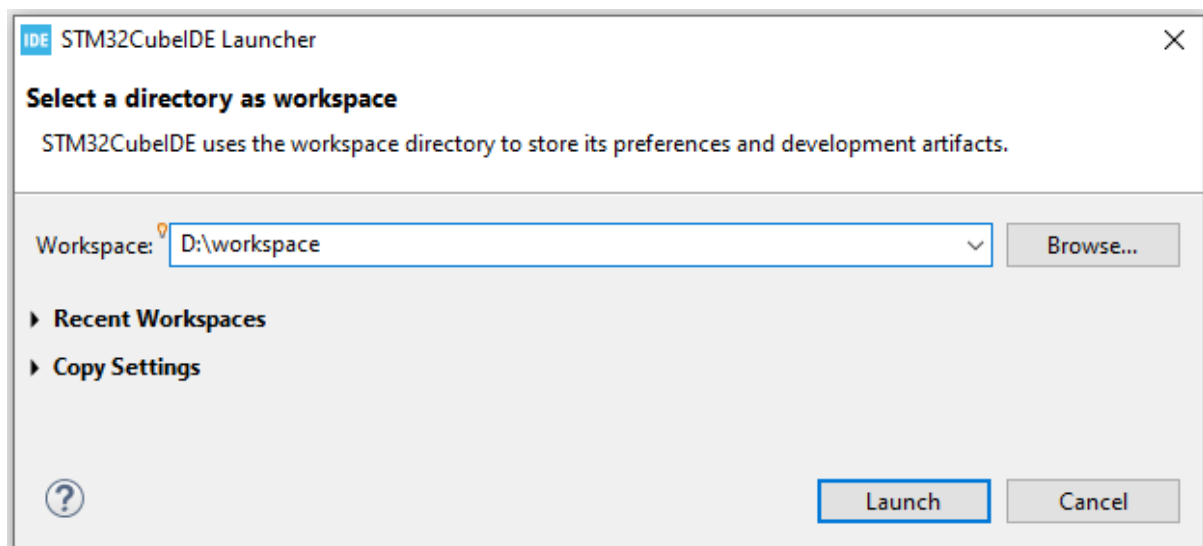
	Part Number	General Description	Software Version	Download	Previous versions
+	STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	1.5.1	Get Software	Select version ▾
+	STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	1.5.1	Get Software	Select version ▾
+	STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	1.5.1	Get Software	Select version ▾
+	STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	1.5.1	Get Software	Select version ▾
+	STM32CubeIDE-Win	STM32CubeIDE Windows Installer	1.5.1	Get Software	Select version ▾

1. Ábra - Az STMicroelectronics honlapján található letöltési táblázat

Forrás: STMicroelectronics

<https://www.st.com/en/development-tools/stm32cubeide.html>

Ha a telepítés befejeződött, nyissuk meg a programot! Az első indításkor meg kell adnunk egy "workspace"-t, vagyis egy mappát ahova alapértelmezetten mentésre kerülnek majd projektjeink.



2. Ábra - Workspace beállítása

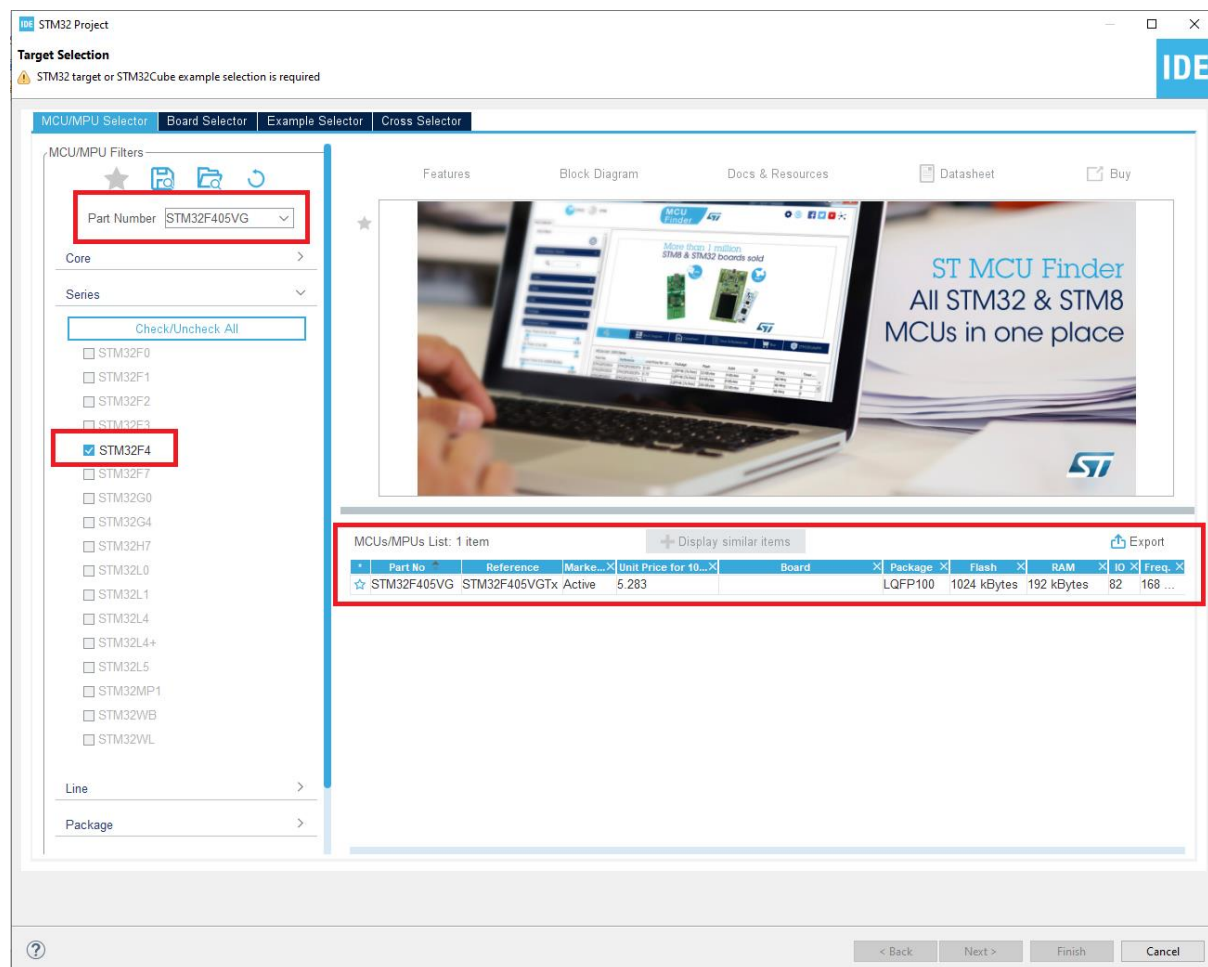
Mindenképpen úgy állítsuk be, hogy az elérési útvonal ne tartalmazzon ékezetes vagy speciális karaktereket, mert az hibás működéshez vezethet.

Ezt követően az *Information Center* fogad bennünket, ahol elérhetjük a fejlesztőkörnyezethez kapcsolódó összes leírást, illetve könnyedén elkezdhetünk egy általunk kiválasztott projekten dolgozni.

ÚJ PROJEKT LÉTREHOZÁSA

Hozzuk létre az első projektünket! Ezt vagy az *Information Center* ablakában található **"Start new STM32 project"** gombra való kattintással tehetjük meg, vagy a **File** menüpont **New → STM32 Project-re** való gombnyomással. Ezt követően előugrik a projekt varázsló. Első lépésben ki kell választanunk az

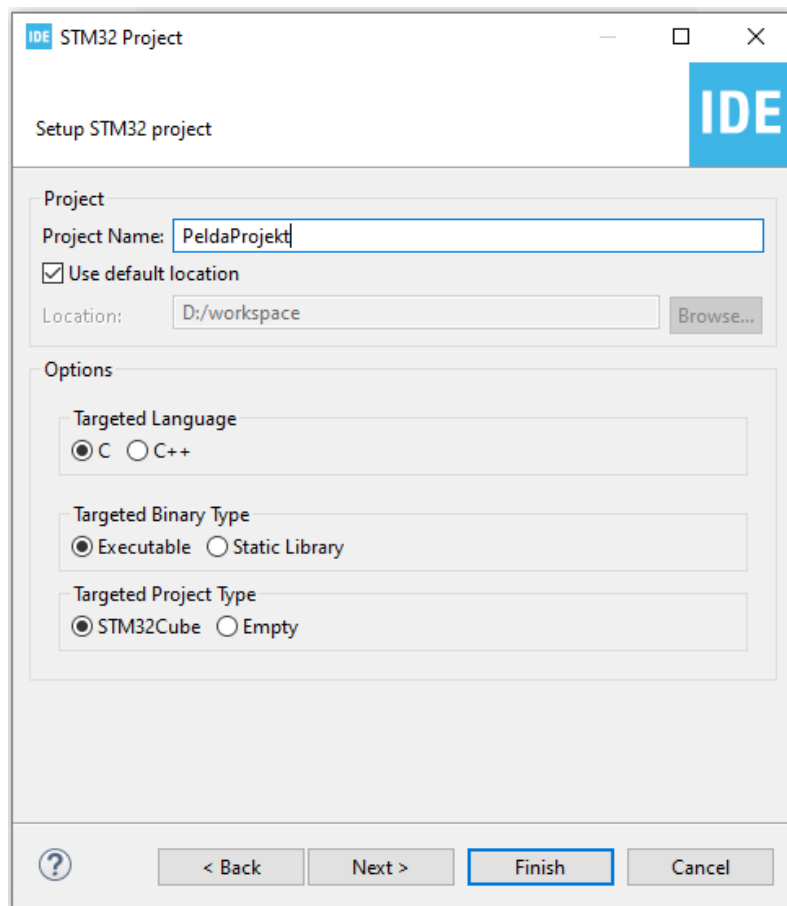
általunk használt mikrokontroller típusát. A tananyaghoz tartozó fejlesztőkártyán egy STM32F405VG mikrokontroller található, így a listából ezt kell kiválasztanunk.



3. Ábra - Target Selector: A használni kívánt mikrovezérlő kiválasztása

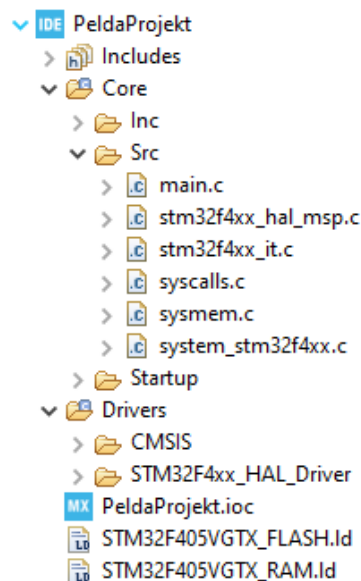
Ha ez megvan, el kell neveznünk a projektet, a neve most legyen egyszerűen *“PeldaProjekt”*, de általában érdemes a projektre jellemző beszédes névvel ellátni, például egy motorvezérlő projekt esetében MotVez stb.

Az elnevezések során mindig kerüljük az ékezetes és speciális karaktereket, a fejlesztői környezet dokumentációjában megtalálható, hogy nem tudja ezeket helyesen kezelni, ezért a projekt elérési útja és a neve is csak ASCII karaktereket tartalmazhat!



4. Ábra - Projekt létrehozásának beállításai

Ügyeljünk arra, hogy az **Options** részben a *C nyelv*, az *Executable*, illetve az *STM32Cube projekt* típus legyen kiválasztva. A **Finish** gombot megnyomva létre is hoztuk az első projektünket. Nézzük meg, mit tartalmaz egy ilyen projekt az alapértelmezetten baloldalt található **Project Explorerben**:



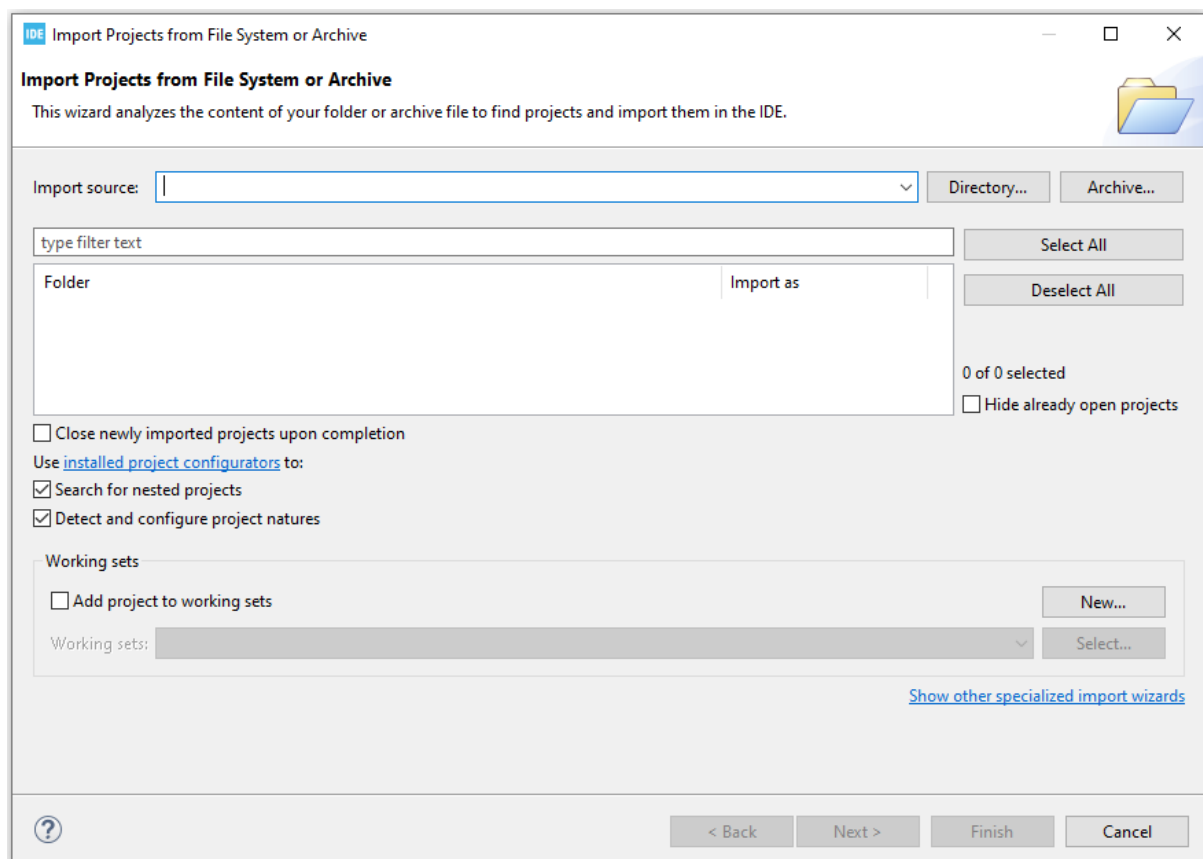
5. Ábra - STM32CubeIDE projekt felépítése

Egy projekt 5 fő elemet tartalmaz.

- **Includes:** Az előző tananyagrészen is említett include-olható fájlok elérési útvonalát tartalmazza.
- **Core:** Itt találhatóak a forrás- illetve fejlécfájlok, az Src mappában megtalálható a main.c, ami tartalmazza a main() függvényt. Ez a programunk futásának belépési pontja.
- **Drivers:** itt találhatóak különböző kiegészítő szoftvercsomagok, mint például a mikrokontrollerhez tartozó HAL library is, erről a következő tananyagrészen lesz részletesebben szó.
- **CubeMX “.ioc” fájl:** Ezt megnyitva végezhetjük el a perifériák, illetve a ki- és bemenetek egyszerű beállítását. Részletesebben később lesz róla szó.
- **Memóriakonfigurációs fájlok:** A fejlesztőkörnyezet által generált konfigurációs fájlok. Itt található a mikrokontroller memóriatérképe és emellett sok egyéb információ is.

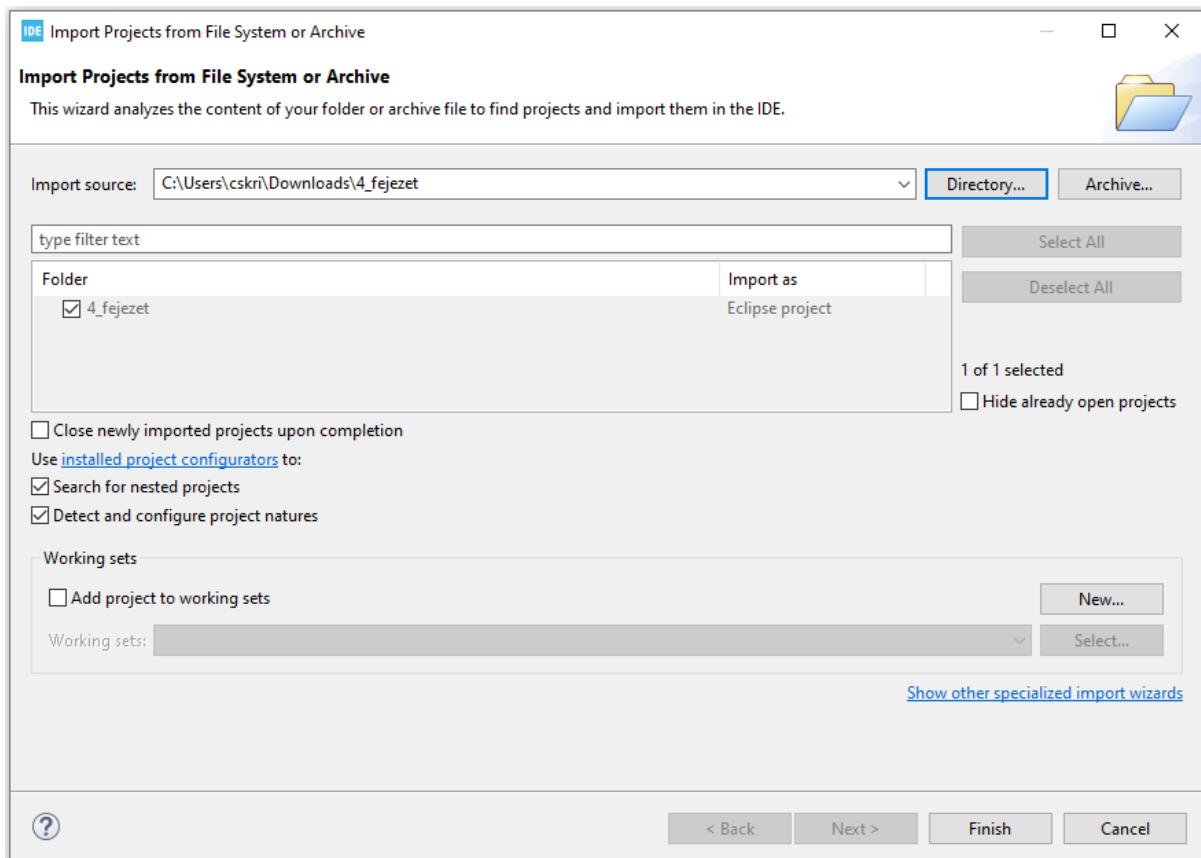
PROJEKT IMPORTÁLÁSA

Lehetőségünk van külső forrásból származó (például letölthető tartalmak, demo projektek stb.) projekteken is dolgozni, azonban ezeket előbb importálnunk kell a fejlesztőkörnyezetbe. Nyissuk le a **File** menüt, majd kattintsunk a **Open Project from File System...** menüpontra! Ha ezt megtettük, akkor a következő ablakot kell látnunk:



6. Ábra - Projekt importálására szolgáló felület

Elsőként ki kell választanunk az importálandó projektet a számítógépen. A projekt lehet mappában, vagy tömörített állományban. Az első esetben a **Directory...** míg a második esetben az **Archive...** gombot válasszuk! Töltsük le a tananyagrészhöz tartozó példaprojektet, majd tetszőleges módon importáljuk a fejlesztőkörnyezetbe! Tömörített esetben az **Archive...** gombra kattintva keressük ki az imént letöltött állományt, míg a másik esetben előbb ki kell csomagolni a letöltött mappát, csak utána tudjuk a **Directory...** gomb megnyomását követően tallózni a fájlrendszer megfelelő helyéről. Mivel mindkettő esetben ugyanúgy bekerül a projekt a fejlesztőkörnyezetbe, ezért mindegy, hogy melyik megoldást választjuk.

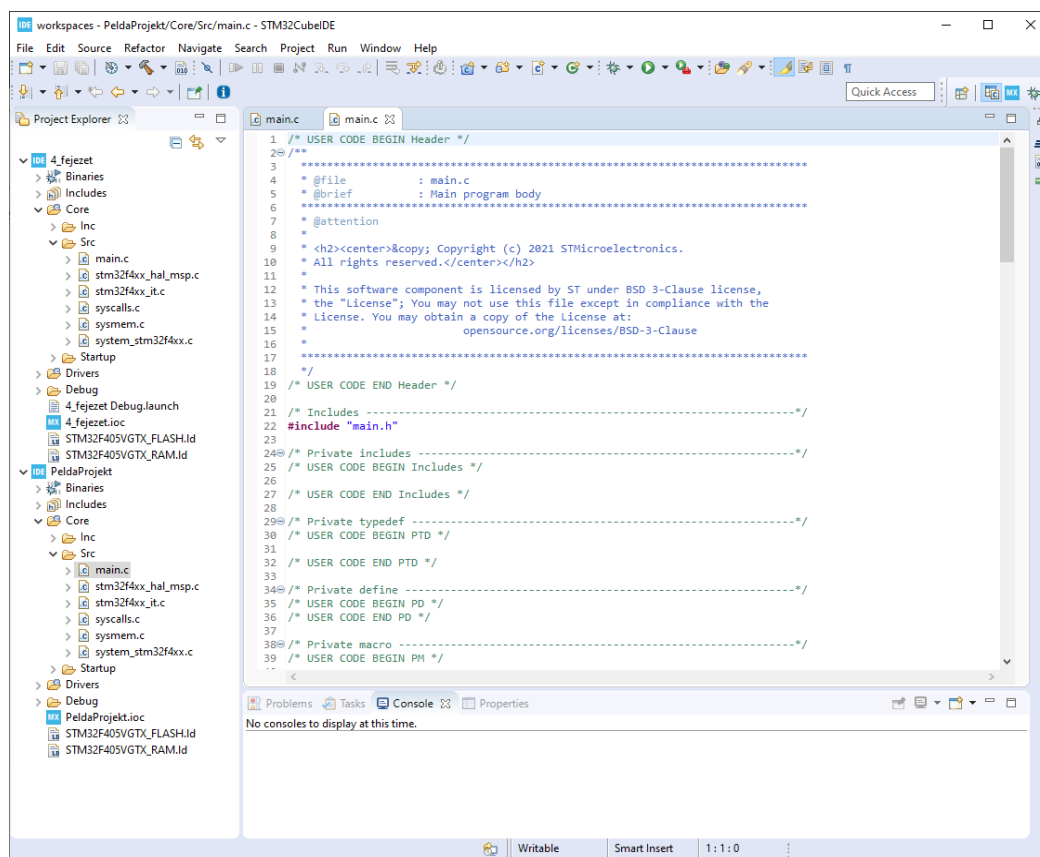


7. Ábra - Importáló felület: Sikeresen megnyitott projektmappa

A sikeres megnyitás után a fenti ablakot kell látnunk. A könyvtár vagy a tömörített állomány kiválasztása után megjelenik a listában az összes benne található projekt, jelen esetben egy projekt van, ennek a *4_fejezet* a neve. Van olyan, hogy nem található a könyvtárban vagy állományban projekt (például mert sérült, vagy mert nem STM32CubeIDE/Eclipse-kompatibilis projekt), ilyenkor ott nem jelenik meg semmi. Előfordulhat olyan helyzet is, hogy megjelenik úgy, mint most, hogy *4_fejezet*, de valamilyen hibaüzenet fogad minket. Leggyakrabban az a probléma, hogy van már vele azonos nevű projekt a workspace-en, ezért nem tudjuk importálni. Ilyenkor le kell ellenőrizni, hogy nem szerepel-e már ez a név a **Project Explorerben**, vagy a workspace mappájában. Ha a tallózás sikeres volt, a **Finish** gombra kattintva bekerül a projektünk a bal oldalt található **Project Explorerbe**, ha lenyitjuk, a korábban már említett projekt struktúrát látjuk.

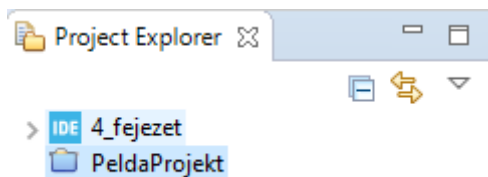
PROJEKT BEZÁRÁSA ÉS TÖRLÉSE

Most, hogy már több projektünk is van a fejlesztőkörnyezetben, foglalkozunk egy kicsit a kezelésükkel. Nyissuk meg a *main.c* fájlt mind a *PeldaProjekt*, mind a *4_fejezet* projektjéből, így kettő különböző, de azonos nevű fájl szerepel előttünk megnyitva.



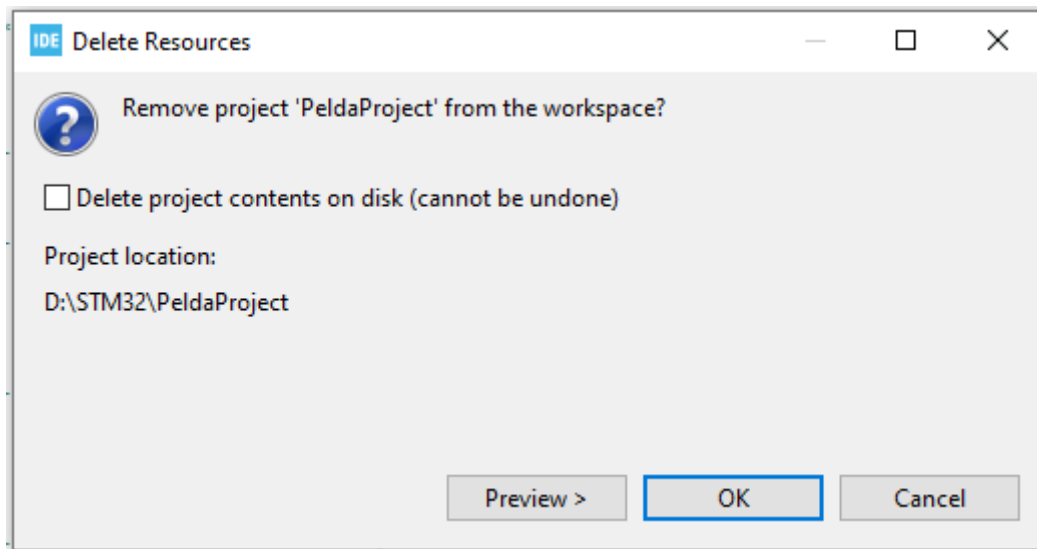
8. Ábra - STM32CubeIDE-ben megnyitott azonos nevű állományok (2x main.c)

Mi is ezzel a gond? Ha több projektünk is meg van nyitva egyszerre, fennáll a veszélye, hogy rossz fájlban kezdünk el dolgozni, rossz projektet fordítunk le, vagy kezdünk el debugolni a névegyezés következtében (ezekről bővebben később), amik időigényes folyamatok is lehetnek. Habár az ablak nevében látható az elérési út, azokat a projekteket mindig zárjuk be, amelyeken éppen nem dolgozunk (egy projekt meg van nyitva, ha a kis STM32CubeIDE ikon található előtte, és be van zárva, ha egy kék mappa, lásd a következő képen). Kattintsunk jobb egérgombbal a *PeldaProjekt* névén, majd válasszuk ki a **Close Project** opciót. Ha ez megtörtént, láthatjuk, hogy nem csak a **Project Explorer**ben változott meg a projekt (nem lehet legördíteni, más az ikon), de bezárult a hozzá tartozó *main.c* fájl is (ilyenkor az összes, a projekthez tartozó megnyitott fájl bezáródik, de a workspace-ről nem tűnik el, nem kell újra importálni).



9. Ábra - Megnyitott és bezárt projekt

Tegyük fel, hogy a *PeldaProjekt*en elvégeztünk minden feladatot, nincs rá tovább szükségünk. Felesleges tehát, hogy a fejlesztőkörnyezet továbbra is tartalmazza azt, töröljük ki! Kattintsunk jobb egérgombbal a projekt nevére, és válasszuk a **Delete** opciót. A következő felugró ablak fog fogadni minket:

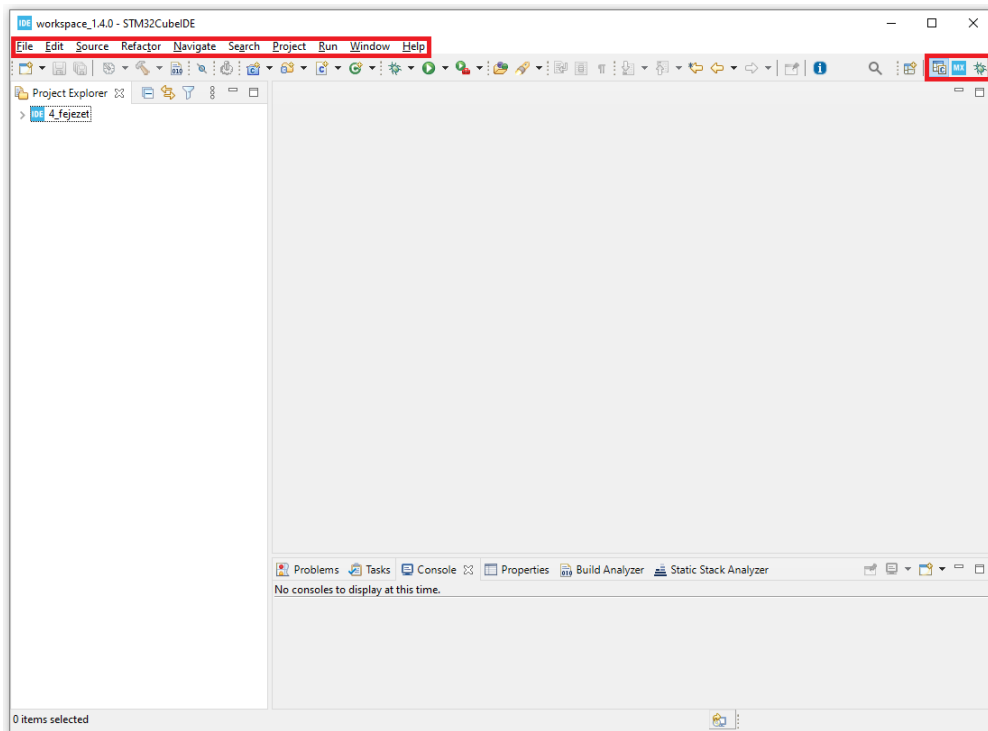


10. Ábra - Projekt törlésének megerősítő ablaka

A pipálható mező (Delete project contents on disk) segítségével dönthetjük el, hogy csak a fejlesztőkörnyezetből, vagy a számítógépről is szeretnénk-e törölni a projekt teljes tartalmát. **Vigyázat: ha egyszer kitöröltük, nem lehet visszaállítani, mint például a lomtárba helyezett elemeket, tehát ez végleges törlést jelent,** így érdemes körültekintően eljárni. Most válasszuk ki a végleges törlés opciót, és hajtsuk végre. Ezek után, ha megnézzük a workspace-nek beállított mappát és a fejlesztőkörnyezetet, mindkettő helyről eltűnt a projekt.

FELHASZNÁLÓI FELÜLET

Most, hogy a projektekkel már tudunk bánni, nézzük is meg jobban a felhasználói felületet, amely alapértelmezetten a következőképpen néz ki:



11. Ábra - STM32CubeIDE általános felhasználói felülete

Menüsor

Kezdjük a fejlesztőkörnyezet ablakának tetején, a menüsor és az ikonok áttekintésével. A menüsor első két eleme a legáltalánosabb, a **File** menüben a szokásos mentés, megnyitás stb. opciókat, míg az **Edit** menüpontban a szerkesztési lehetőségeket találjuk meg, ezek például a visszavonás, visszaállítás, másolás, beillesztés.

A következő a **Source** menüpont. Az első részében kommentek beállítására van lehetőség, például, ha kijelölünk egy sort, a **Toggle Comment** megnyomására a fejlesztőkörnyezet kommentté alakítja. Továbbá itt van lehetőség a kód behúzásának (beljebb vagy kijebb) beállítására is, néhány egyéb kódformázási beállítás mellett.

Kitekintés:

KOMMENTEK

Mik egyáltalán azok a kommentek? A válasz egyszerű: minden olyan dolog, ami a valóságban nem kerül bele a lefordítandó kódba (az előfordító kiszűri őket), csupán a fejlesztő dolgát könnyítő megjegyzések. Rövid kommentet a `//` jelek mögött, a több soros megjegyzéseket pedig `/* */` jelek között találhatunk. De miért van erre szükség? Ha bármilyen kódot írunk, legyen az egyszerű összeadás, vagy egy bonyolult LCD kijelző kezelés, az egy idegen szem számára nehezen érthető, még akkor is, ha jártas a területen (vagy akár saját magunknak, ha hosszú kihagyás után térünk vissza a projekthez). Viszont, ha gondosan ellátjuk kommentekkel a munkánkat, akkor nem kell sorról sorra fejtegetni, hogy mit is csinál a kódunk, hanem elegendő elolvasni a kommenteket. Erre tipikusan jó példa a tananyag rész példakódjának egy része:

```

*RCC_AHB1 = 0x00100008;           // Periféria regiszter beállítása
*GPIO_MODDER_addr = 0x00004000;    // PD7-es GPIO láb kimenetre állítás
*GPIO_OTYPER_addr = 0x00000000;    // Minden GPIO push-pull módra állítása
*GPIO_OSPEEDR_addr = 0x00000000;   // Alacsony sebesség
*GPIO_PUPDR_addr = 0x00000000;     // Nincs felhúzó és lehúzó ellenállás
*GPIO_IDR_addr = 0x00000000;       // Bemeneti regiszter 0-ra állítása
*GPIO_ODR_addr = 0x00000000;       // Kimeneti regiszter 0-ra állítása
*GPIO_BSRR_addr = 0x00000000;      // Nincs S-R flip flop a kimeneten
*GPIO_LCKR_addr = 0x00000000;      // Nincs lezárás
*GPIO_AFR_L_addr = 0x00000000;     // Alap funkcionalitás
*GPIO_AFR_H_addr = 0x00000000;     // Alap funkcionalitás

```

Kódrészlet a tananyagrészhöz tartozó projektből

Biztos vagyok benne, hogy van olyan okos fejlesztő, aki ezeket a sorokat első pillantásra meg tudja fejteni a kommentek nélkül is, de lássuk be, sokkal beszédesebbek a // jel mögötti részek. Ami a legfontosabb: mindig írjunk kommenteket, hiszen ezzel nem csak mások, de a saját munkánkat is megkönnyítjük majd. (Át lehet esni a ló túloldalára is! Nem érdemes teljesen egyértelmű megjegyzésekkel illetni a könnyen értelmezhető részeket, mert zavaró lesz a kód olvasása közben.)

A **Refactor** menüpont alatt a kódunk egyszerű és gyors módosítására van lehetőség, kezdve az átnevezésektől az egyes kifejezések vagy függvények áthelyezéséig. A **Navigate** menüpontban található elemek segítenek egy nagyobb kódban való eligazodáshoz. Itt tudunk a kód egyik részéről a másikra átmenni, vagy beléphetünk a függvények deklarációjába is. A **Search** menüben keresni tudunk, legyen az fájl, egy szó, vagy valamilyen kifejezés. A **Project** fülben a projekt **“Build”**-elési (fordítás) lehetőségeit tudjuk kipróbálni, illetve az ezzel kapcsolatos beállításokat elvégezni. A **Run** menüben a debuggolási funkcióhoz kapcsolódó parancsokat találhatjuk meg, ezekről még később lesz bővebben szó.

Nézetek

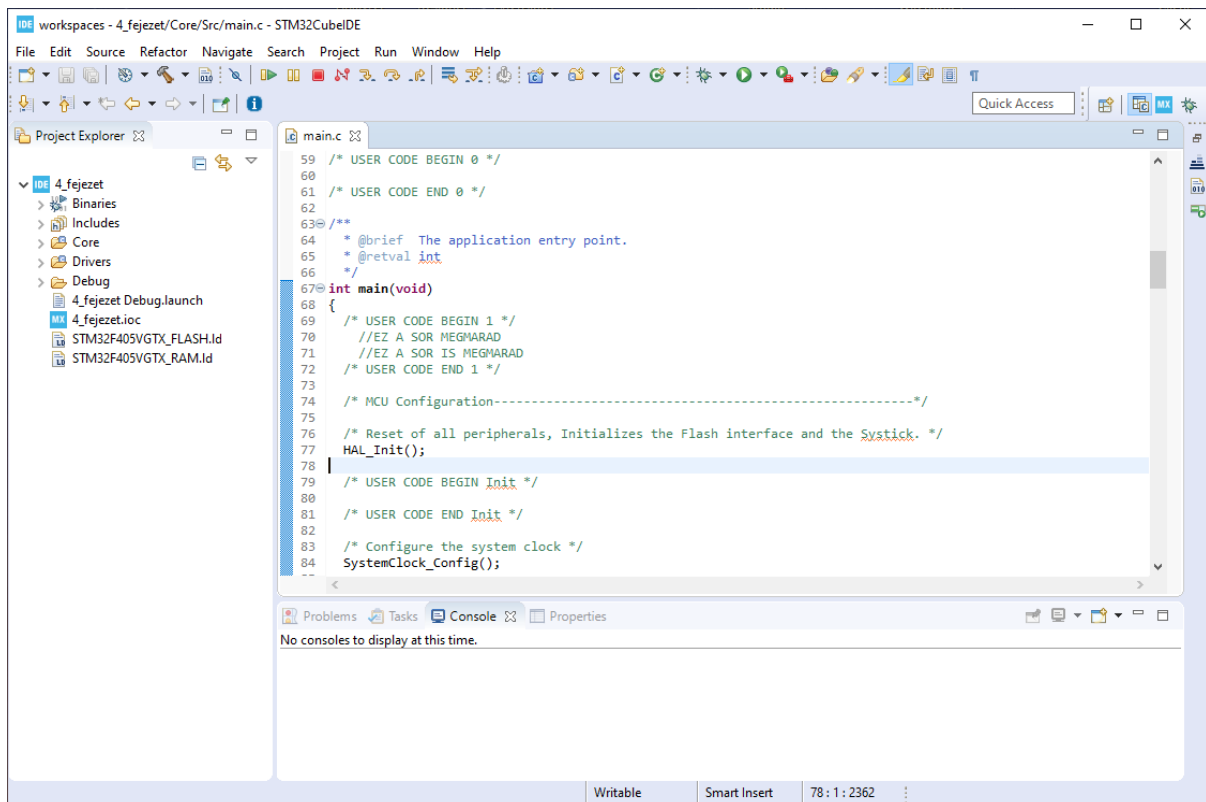
A fejlesztőkörnyezet alapvetően több nézetből (kisebb ablakok) és perspektívából épül fel. Ezeket egyéni preferencia szerint helyezhetjük el, zárhatjuk be vagy nyithatjuk meg. Az STM32CubeIDE alapbeállításként 3 perspektívával rendelkezik (ezeket a jobb felső sarokban láthatjuk), ezek a kisebb ablakok elhelyezkedését és méretét befolyásolják, de leginkább azt, hogy melyik legyen megjelenítve és melyik nem.



12. Ábra - C/C++ - CubeMX - Debug perspektívák

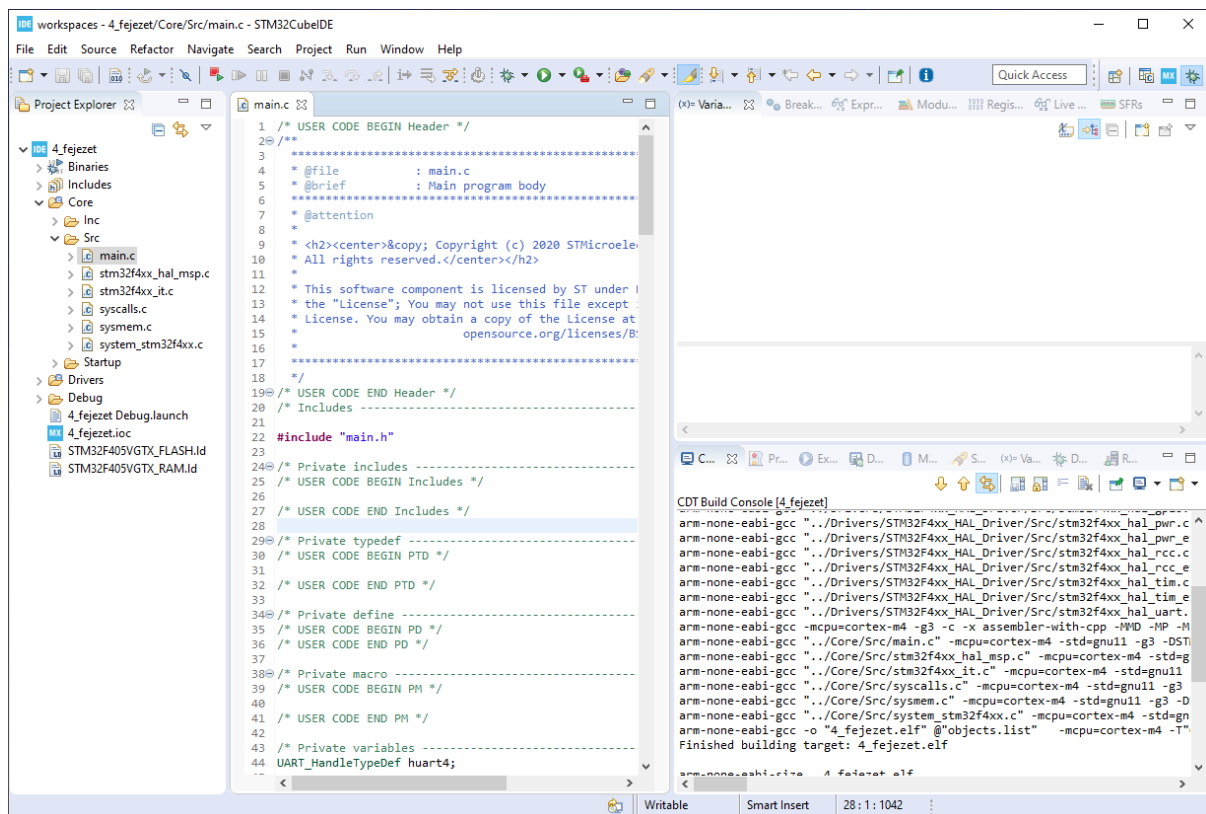
Az első a C/C++ nézet, ez alapvetően a kód írásra/buildelésre lett kitalálva. Itt láthatjuk baloldalt a **Project Explorer**-t, középen a megnyitott fájlokat, alul pedig több, a buildeléssel és a projekttel kapcsolatos információs ablakot. Ezek természetesen ízlés szerint módosíthatóak, de nem kell

megijedni akkor sem, ha valamelyiket véletlenül bezárnánk, hiszen a **Window** menü **Show View** alpontjából bármikor újra meg tudjuk azt nyitni.



13. Ábra - A C/C++ perspektíva felépítése

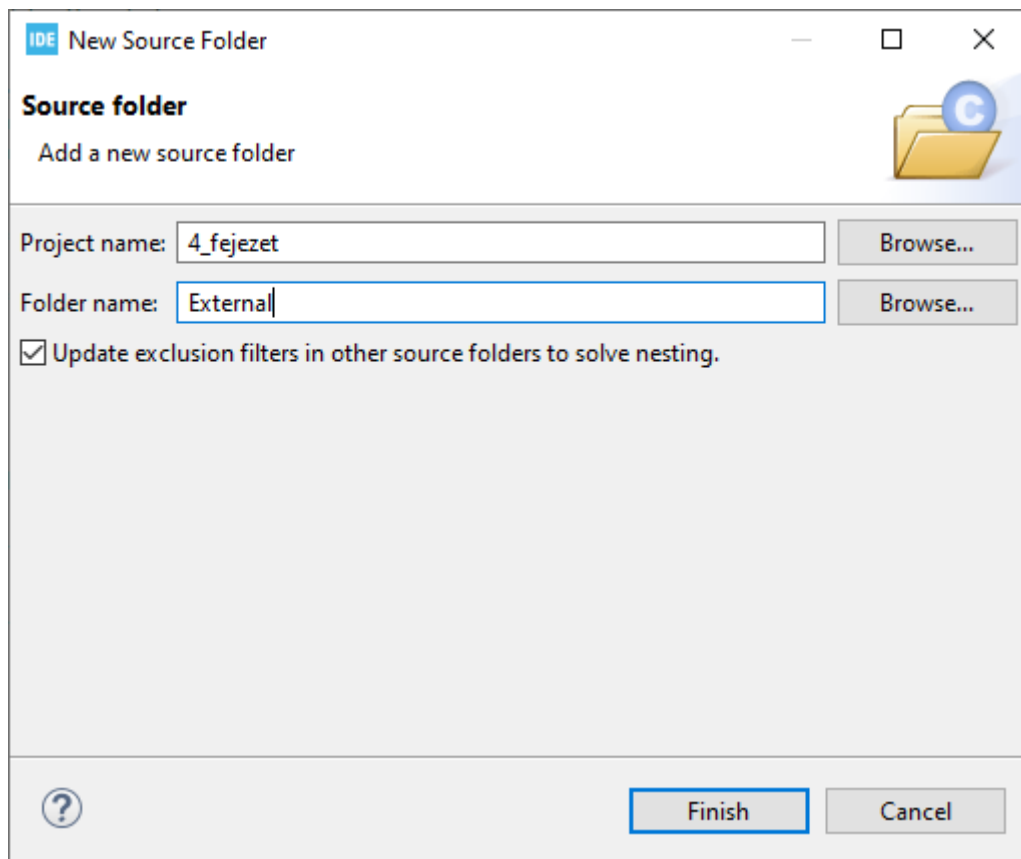
Az alul található információs panelen a **Console** fülön találkozhatunk a fejlesztőkörnyezet által végzett folyamatok leírásával (például, hogy éppen melyik fájlokat fordítja le). Nézzük meg, mit is ír ki, ha megnyomjuk a felső ikonsorban található kis kalapácsot (Build opció, ha nem elérhető, először kattintsunk a forrásfájlra, vagy a projektre a Project Explorerben). Láthatjuk, hogy rengeteg üzenet érkezik be, ezeket a fordítóprogram írja ide (ez bele van építve a fejlesztőkörnyezetbe).



15. Ábra - Debug perspektíva

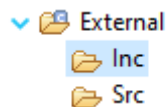
Fájlkezelés

Az előző tananyagrészen már esett szó arról, hogy a programunkat érdemes az áttekinthetőség érdekében strukturáltan létrehozni, azaz a kódot nem mind a *main.c* fájlba írni, hanem forrás- (.c) és fejlécfájlokra (.h) szétbontani. Ezt kétféleképpen is megtehetjük. Ha a projekt mappájának Core almappájában helyezük el a .c (Src) és .h (Inc) fájlokat, akkor elegendő a *main.c*-ben egy include direktívát elhelyezni. Azonban, ha az általunk készített forrásfájlokat szeretnénk szétválasztani a fejlesztőkörnyezet által generált elemektől, érdemes azokat külön könyvtárba elhelyezni. Hozunk létre a projekten belül egy új mappát (jobb klikk a projekten, **New** → **Source Folder**), itt ügyeljünk rá, hogy nem a sima Folder, hanem mindenképpen a Source Folder opciót válasszuk! Adjuk neki most az *External* (külső) nevet!



16. Ábra - Új Forrás mappa létrehozása

Ezen belül hozzunk létre egy *Src*, és egy *Inc* almappát, előbbiben a .c, míg utóbbiban a .h állományokat fogjuk tárolni.



17. Ábra - Ajánlott mappaszerkezet

Hozzunk létre az *Src* mappában egy *pelda.c* nevű fájlt (jobb klikk a mappán, **New** → **Source File**). Járjunk el hasonlóan az *Inc* mappában, ott egy **Header File**-t hozzáadva, amit *pelda.h*-nak nevezünk el. Ha megnyitjuk a .c fájlt, láthatjuk, hogy teljesen üres. Helyezzük el benne a `#include "pelda.h"` direktívát (ezzel mondjuk meg a fordítónak, hogy hol kell keresni a deklarációkat). Másoljuk be ebbe a fájlba a következő kódrészletet:

```
int peldaFuggveny() //Példa függvény definiálása
{
    //Itt csinálunk valamit

    return 0;
}
```

A *pelda.h* fájlba pedig a következő sort illesszük be a 11. sorba:

```
int peldaFuggveny(); //Példa függvény deklarálás
```

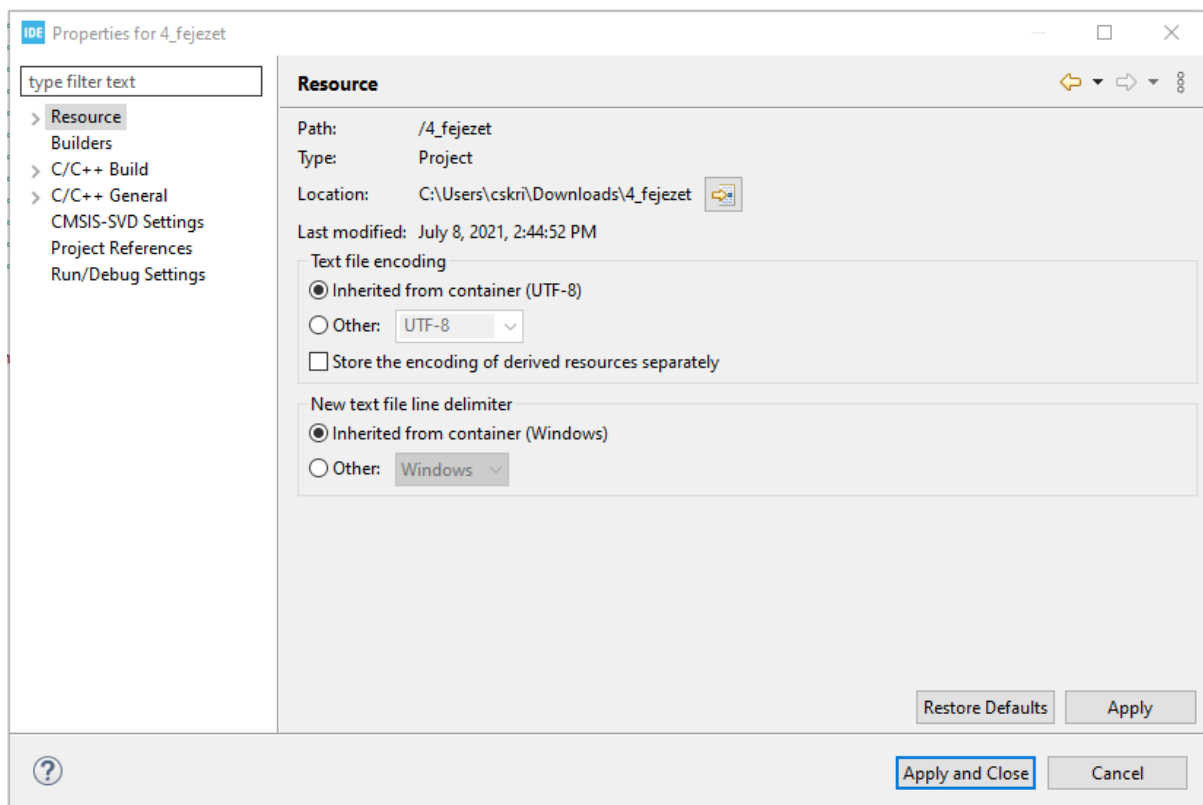
Ezzel létrehoztuk mind a definíciót (előbbi), mind a deklarációt (utóbbi) a függvényünkhöz. A *main.c* fájlban helyezzük el a header fájlhoz tartozó direktívát, majd hívjuk meg ezt a függvényt a *main()* függvényen belül:

```
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "pelda.h"
/* USER CODE END Includes */

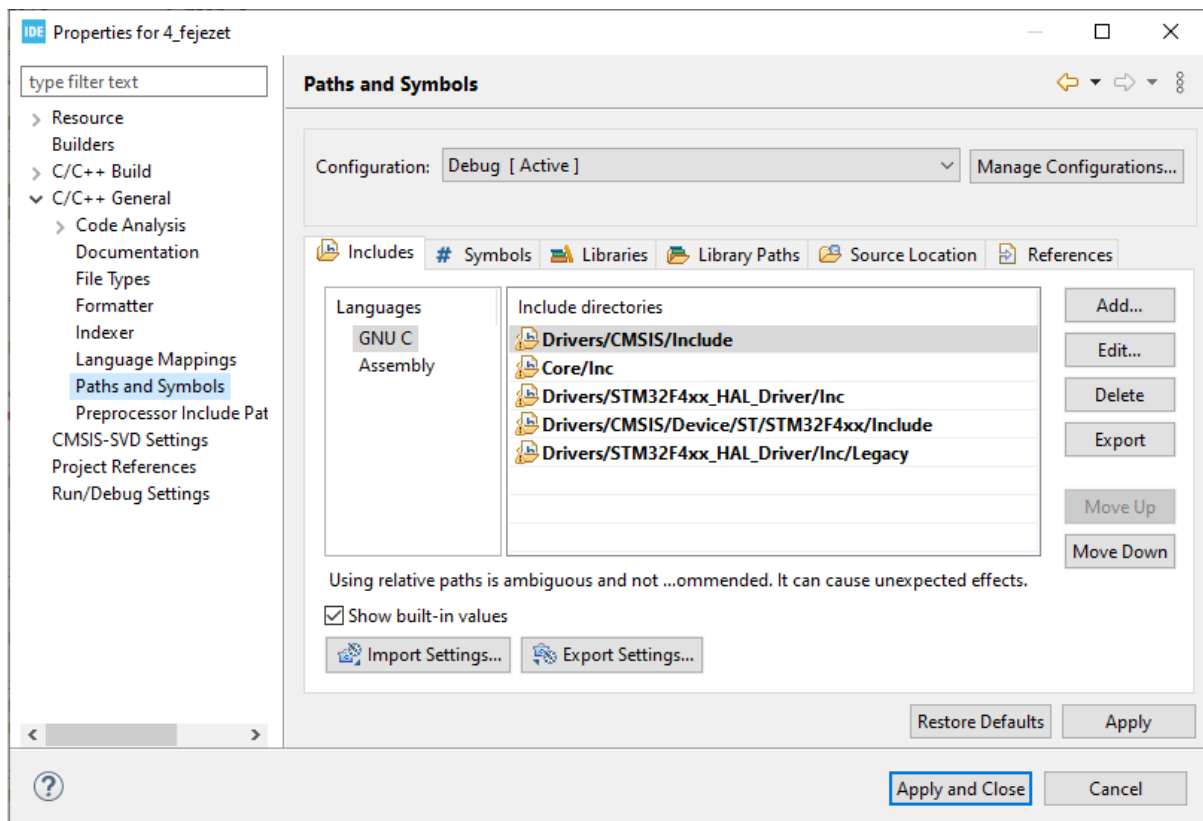
int main(void)
{
    /* USER CODE BEGIN 1 */
        peldaFuggveny();
    /* USER CODE END 1 */
}
```

Ezt követően hiába próbálnánk meg buildelni a programunkat, hibát jelezne, mivel nem tudja, hol kell keresni a *peldaFuggveny()*-t. Meg kell mutatnunk a fordítónak, hogy hol találhatja meg a függvényünk deklarációját, erre szolgál a `#include` direktíva. Mivel ezt már megtettük, már csak azt kell megmutatnunk, hogy az include-olt fájlt hol találja. Kattintsunk jobb egérgombbal a projekt nevén, majd válasszuk a **Properties** opciót (ugyanaz elérhető a felső menüsor **Project** → **Properties** fülből is). A következő ablak fogad majd bennünket:



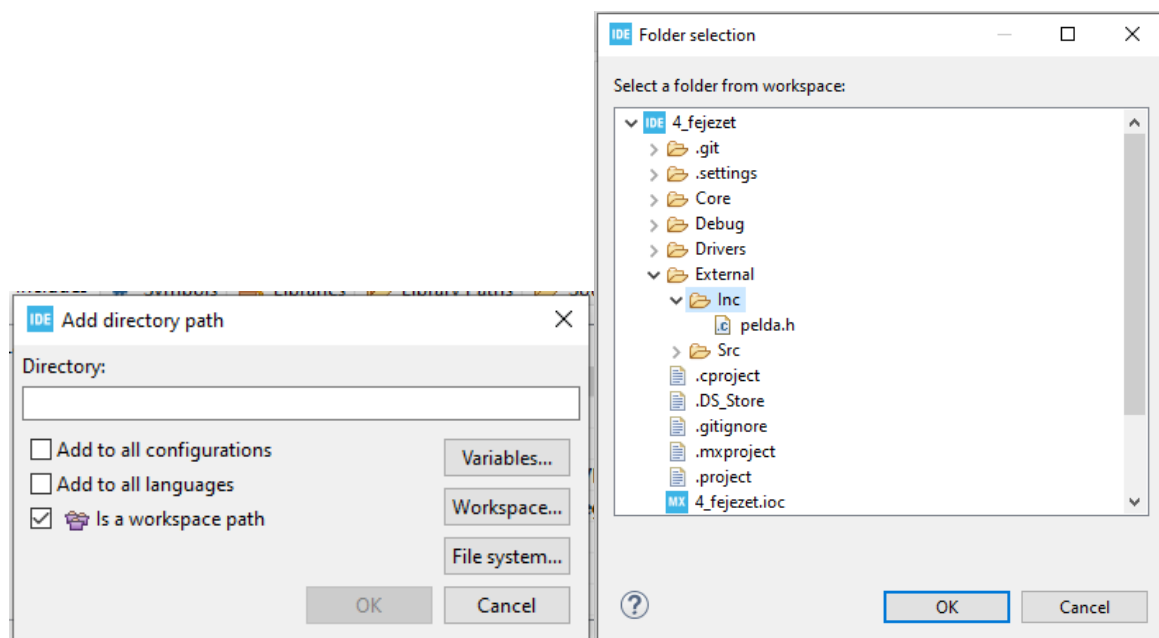
18. Ábra - Project → Properties ablak

Gördítsük le a **C/C++ General** opciót, azon belül pedig válasszuk a **Paths and Symbols**-t! Itt az **Include** fül alatt láthatjuk, hogy a fordító milyen mappában keresi a header fájlokat.



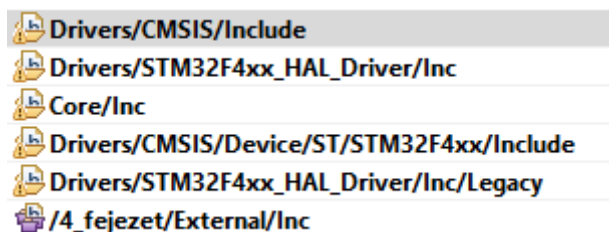
19. Ábra - Include-ok kezelése a Properties ablakban

Adjuk hozzá az általunk készített header fájlunkat tartalmazó mappát: Kattintsunk az **Add...** gombra!



20. Ábra - Könyvtár hozzáadása az Include-ok listájához

Pipáljuk be, hogy **“Is a workspace path”**, avagy a workspace-ben belül található a mappa, majd a Workspace gomb megnyomásával válasszuk ki a projektben belül az *External/Inc* mappát. Ezt fogadjuk el, és láthatjuk, hogy be is került a mappák közé az általunk megadott hely.



21. Ábra - Sikeresen hozzáadott Include könyvtár

Nyomjuk meg az **Apply and Close** gombot, a felugró ablakokban pedig nyomjunk **Yes-t** (ezzel automatikusan újrabildelehetjük az index-et). Láthatjuk, hogy a fordítás hiba nélkül megtörténik, sikeresen szeparáltuk a saját állományainkat, és megmutattuk a fordítónak, hogy hol keresse azokat.

Beállítások

Az előző részben egészen sok szó esett a fordításról (Build), és párszor el is végeztettük ezt a folyamatot a fejlesztőkörnyezettel. Ez időigényes, a **Console** ablakban fordítás után láthatjuk is, hogy az aktuális buildelés mennyi időbe telt, például az alább látható esetben **413 ms**.

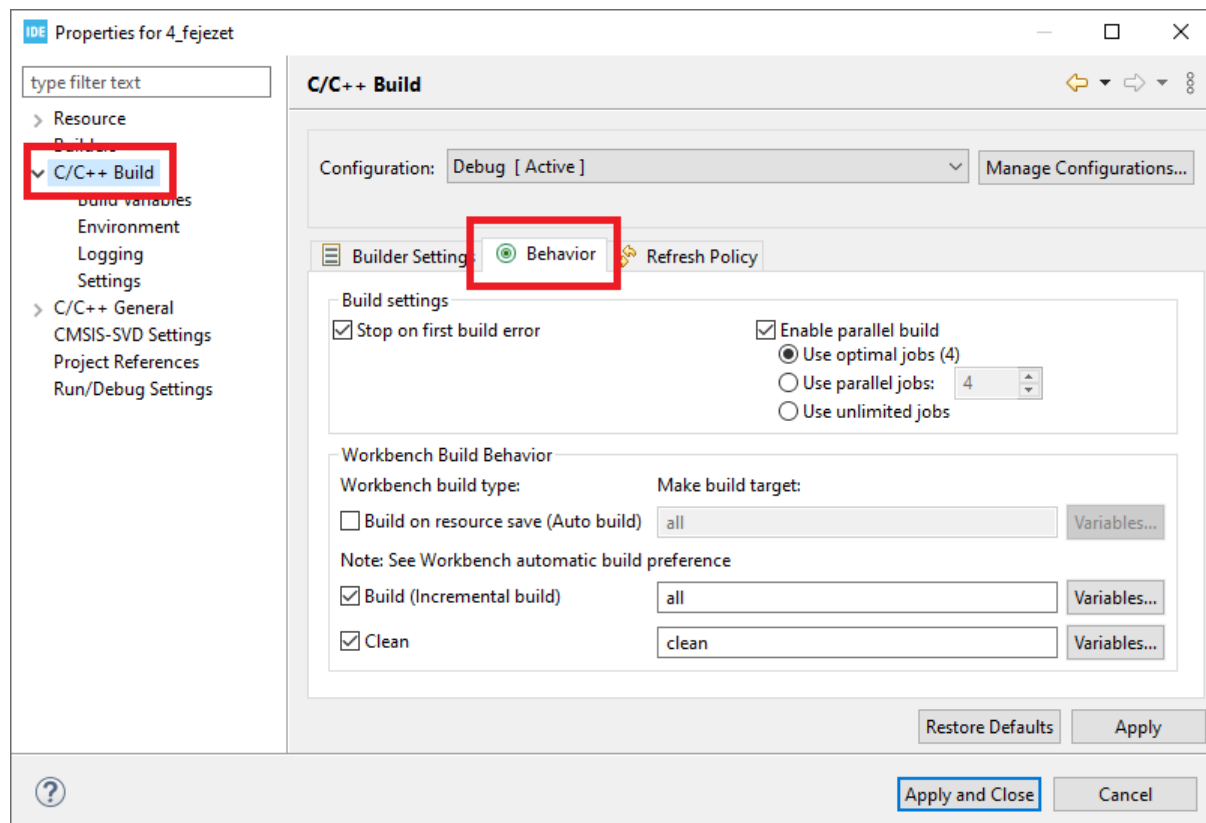
```

CDT Build Console [4_fejezet]
01:03:12 **** Incremental Build of configuration Debug for project 4_fejezet ****
make -j4 all
arm-none-eabi-size 4_fejezet.elf
  text  data  bss  dec  hex  filename
 8148   20  1636  9804  264c 4_fejezet.elf
Finished building: default.size.stdout

01:03:12 Build Finished. 0 errors, 0 warnings. (took 413ms)
  
```

22. Ábra - Lefutott fordítás és az alatta eltelt idő

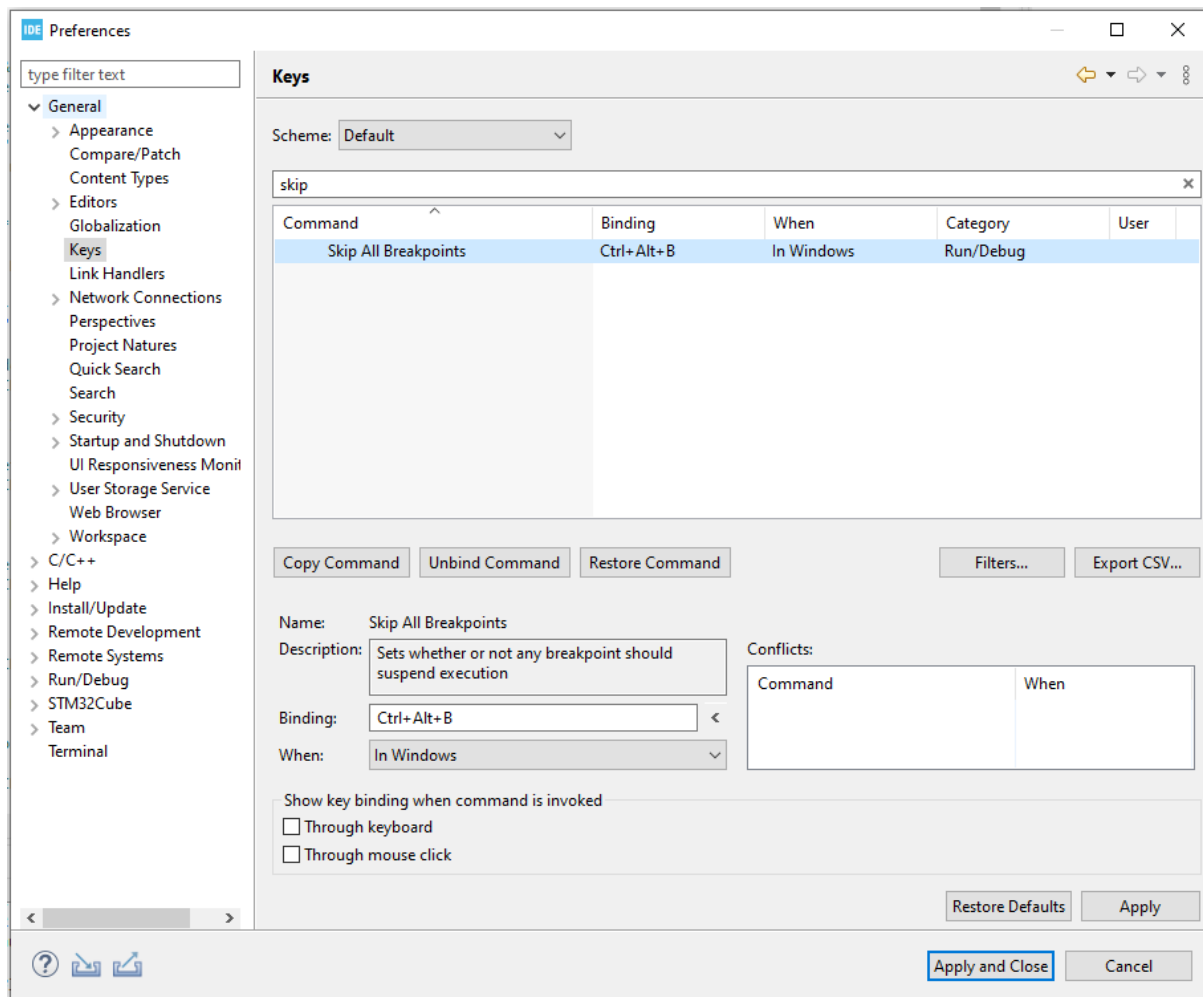
Ez most kevesebb, mint fél másodperc, de sokkal nagyobb projektek esetében (ahol például sok külön fájl van, és azok akár több ezer sorosak) a kód bonyolultságával, méretével arányosan nő a fordítási idő. A fordítási folyamat felgyorsítására és optimalizálására van beállítás a fejlesztőkörnyezetben, hogy a program egyszerre párhuzamosan több folyamatot is végezzen. Mindenképpen használjuk ezt a lehetőséget, mert rengeteg időt megspórolhatunk magunknak, ha nem kell a fordításra sokat várni. Nyissuk meg újra az előbb is látott **Project → Properties** ablakot. Kattintsunk a **C/C++ Build** opcióra, ott pedig a **Behavior** fülre:



23. Ábra - Optimális fordítási beállítások a Project → Properties ablakban

Győződjünk meg róla, hogy az **Enable Parallel build** opció be van pipálva, és a **Use optimal jobs** opció van kiválasztva, ahogy a képen is látható.

Mivel a fejlesztőkörnyezet alapvetően nem a magyar nyelvhez készült, ezért előfordulhatnak az angol és magyar billentyűzetkiosztásából adódó konfliktusok. Például a magyar billentyűzeten a “{” írásához az **AltGr + B** billentyűkombinációt kell megnyomni, míg az angol nyelvű eszközön erre külön gomb szolgál. A probléma abból ered, hogy a megadott billentyűkombináció alapértelmezetten az STM32CubeIDE-ben egy gyorsbillentyű, így nem tudunk kapcsos zárójelet írni, ami elég fontos lenne nekünk, hiszen ez szerves részét képezi a programozásnak. Természetesen van erre megoldás, nézzük is meg most ezt. Válasszuk ki a felső menüsorból a **Window → Preferences** menüpontot, majd a felugró ablakban válasszuk ki a **General** legördülő menüből a **Keys** opciót! A keresőbe írjuk be, hogy “Skip All”, így meg is találjuk az átállítandó parancsot.



24. Ábra - Ctrl+Alt+B billentyűk felszabadítása

Válasszuk ki, majd kattintsunk az **Unbind Command** gombra! Az **Apply and Close** gomb megnyomását követően próbáljuk ki, most már tudnunk kell “{” karaktert írni. :)

HASZNOS TRÜKKÖK

Egy adott projekt fejlesztése közben forrás- és fejlécfájlok komplex rendszerével dolgozunk, így az egyes információk nem mindig egyből a szemünk előtt, hanem sokszor más mappában és fájlban található. Mivel ezeket kikeresgélni (főleg úgy, hogy azt sem tudjuk, hol keressük) sokáig tarthat, a fejlesztőkörnyezet rendelkezik több hasznos gyorsbillentyűvel, amiket nagyon gyakran használunk is. Térjünk vissza a korábban létrehozott *peldaFuggvény()* meghívásához a *main.c* állományba! Tegyük fel, hogy ezt a függvényt nem mi írtuk, így nem is tudjuk, hogy mit csinál valójában, vagy a projekten belül melyik fájlban van definiálva. Ha belekattintunk a függvény nevébe, majd megnyomjuk az **F2**-t, egy lebegő ablakban megjelenik előttünk a függvény definíciója, így könnyedén meg tudjuk ismerni a működését. Ilyenkor nem közvetlenül a függvény definícióját mutatja meg, hanem ha van előtte komment, akkor a definíció helyett a megjegyzéssel kezdődik a megjelenítés, mely általában sokat segít az értelmezésben.

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    peldaFuggveny();
    /*
    int peldaFuggveny()
    {
        // Itt csinálunk valamit
        return 0;
    }
    */
    /* USER CODE END Init */
}
```

25. Ábra - F2 gomb működése függvényen

Igen, de mi van akkor, ha nekünk ez a függvényműködés nem felel meg? Kattintsunk újra a függvény nevére, de most az **F3** billentyűt nyomjuk meg! A fejlesztőkörnyezet azonnal megnyitja nekünk a függvényt tartalmazó fájlt, és ki is jelöli számunkra a függvény első sorát. Ugyanezt a működést érhetjük el, ha Ctrl lenyomása mellett kattintunk a függvény nevére.

Ezek mind olyan esetek, ahol a függvényünk már meg van hívva. Honnan tudjuk, hogy ez a függvény egyáltalán létezik, és meg lehet hívni? A fejlesztőkörnyezet erre is kínál megoldást.

A main.c fájlban kezdünk egy új sort a *peldaFuggveny()* hívása alá, és írjuk be, hogy *“pelda”*!

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    peldaFuggveny();
    pelda
    /* USER CODE END 1 */
}
```

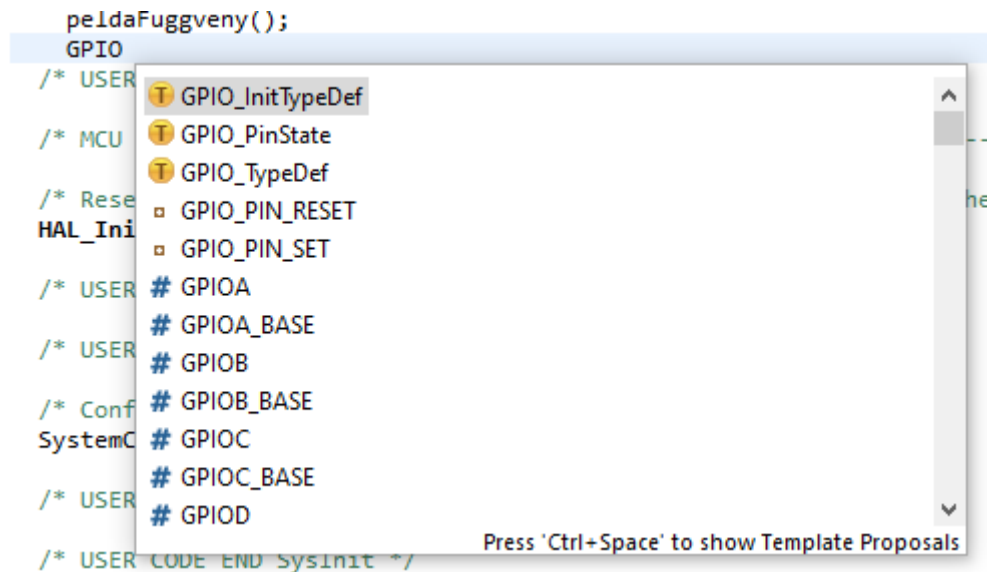
26. Ábra - Gépelés támogatása Ctrl + Space-szel 1.

Ha a *“pelda”* szó végén (nem szükségszerű egész szavakat beírni, akár az első pár betű után is megtehetjük ezt) lenyomjuk a **Ctrl + Space** billentyűkombinációt, a fejlesztőkörnyezet automatikusan kitölti a mezőt a pelda kifejezéshez illő végződéssel:

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    peldaFuggveny();
    peldaFuggveny()
    /* USER CODE END 1 */
}
```

27. Ábra - Gépelés támogatása Ctrl + Space-szel 2.

Ez arra az esetre igaz, ha csak egy darab *“pelda”* szót tartalmazó függvényünk/változónk van. A következő sorba írjuk be, hogy *“GPIO”*, majd újra nyomjunk **Ctrl + Space**-t!

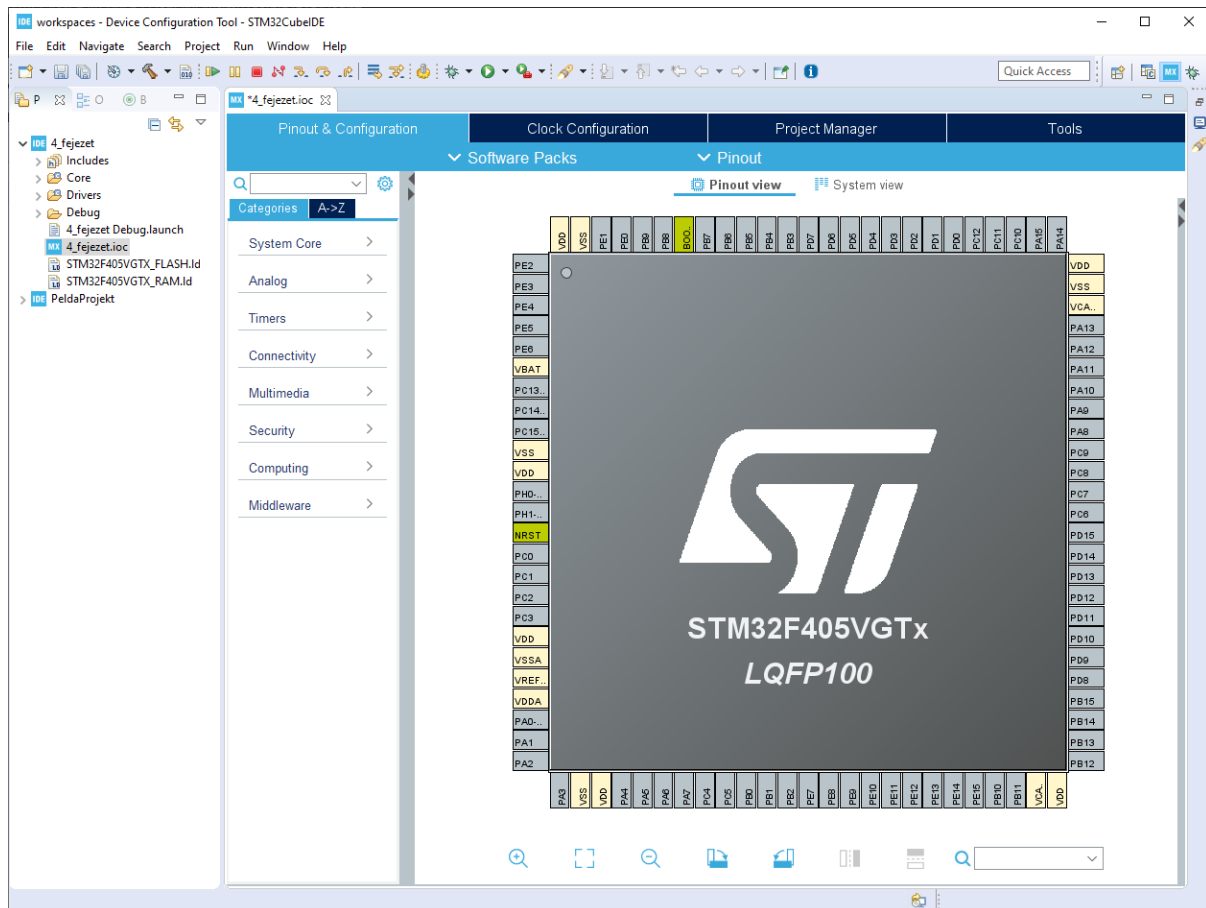


28. Ábra - Gépelés támogatása Ctrl + Space-szel 3.

Láthatjuk, hogy a fejlesztőkörnyezet nagyon sok opciót felkínál, ha most elkezdénénk tovább gépelni, aszerint szűkülne a felajánlott elemek listája (itt látszik a találat típusa is, például typedef, enum, define direktíva stb., ez is sokat segít annak eldöntésében, hogy most melyikre is van szükségünk). Ez a funkció azért működik ilyen jól, mert mind a fejlesztők, mind a könyvtárak elnevezési konvenciókat használnak. Például, ha valami GPIO-val kapcsolatos dolgot akarunk csinálni, akkor az ahhoz szükséges függvény "GPIO_"-val fog kezdődni, azért tudjuk ilyen könnyen megtalálni. Ez a funkció a fejlesztési folyamatot rendkívül megkönnyíti, hiszen elegendő a kifejezések kezdetét ismerni, és az alapján már elő tudjuk hozni a számunkra megfelelőt.

CUBEMX

Korábban a projekt felépítésénél láttuk a `.ioc` fájlt, ami a konfigurációhoz használható, most nézzük meg kicsit közelebbről. Kattintsunk duplán a `.ioc` fájlra, ezzel megnyílik a projekthez tartozó CubeMX perspektíva, ahol lehetőségünk van a mikrokontroller alapbeállításainak elvégzésére, illetve a perifériák felkonfigurálására.



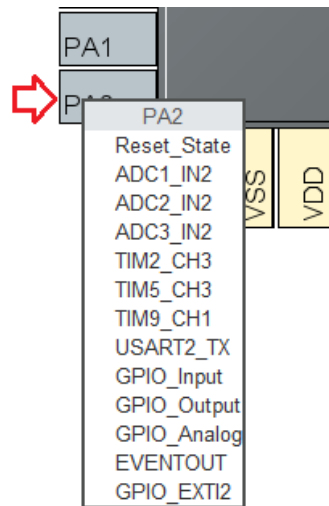
29. Ábra - A CubeMX felhasználói felülete

Tekintsük át a főbb nézeteket, amik között a fenti kék részen lévő gombokkal tudunk navigálni!

PINOUT & CONFIGURATION

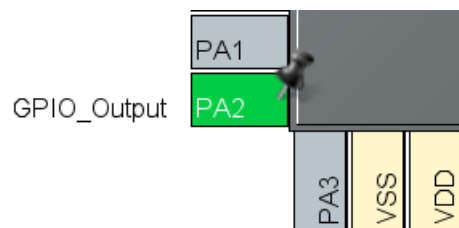
A fő nézetek közül az első a **Pinout & Configuration**, ami a lábkiosztás és konfiguráció végrehajtására szolgál. A középen lévő szürke alakzat magát az általunk használt mikrokontrollert jelképezi, ezen megtalálható a mikrokontroller típusa (esetünkben STM32F405VGT), illetve a mikrokontroller tokozása is (LQFP100, erről a rövidítésről egy későbbi tananyagrészen lesz szó). A sok kis kiálló téglalap pedig a kivezetéseket szimbolizálja (ha megnézzük, a valóságban is így néz ki az eszköz felülnézetben). Az egérrel tudjuk mozgatni, a görgővel pedig közelíteni és távolítani. Ha ránagyítunk a kivezetésekre, láthatjuk, hogy feliratozva is vannak. Ezek az elnevezések különböztetik meg őket, így könnyebb rájuk hivatkozni. A kivezetések portokba vannak rendezve, a nevükben található betű is ezt jelképezi (például a PA2 azt jelenti, hogy P: Port, A: A azonosítójú port, 2: 2. kivezetés a porton).

Alapvetően a mikrokontroller összes lába Reset állapotba van konfigurálva, azaz semmilyen funkciót nem látnak el. Kattintsunk rá bal egérgombbal a példa kedvéért a PA2 kivezetésre!



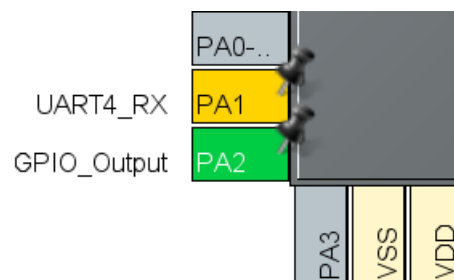
30. Ábra - Egy kivezetés lehetséges beállításainak listája

A fent látható képet kell kapjuk, ahol egy legördülő listából kiválaszthatjuk, hogy milyen funkciót kívánunk beállítani a kivezetésre (alapból *Reset_State* állapotban van). Az, hogy a különböző opciók mit jelentenek és mire valók, egy későbbi tananyagrészen kerül bővebben kifejtésre. Kattintsunk valamelyik opcióra a legördülő menüben! Láthatjuk, hogy a kiválasztott funkció hozzá is rendelődött a kivezetéshez, mely az ábrán zölddel lett megjelölve.



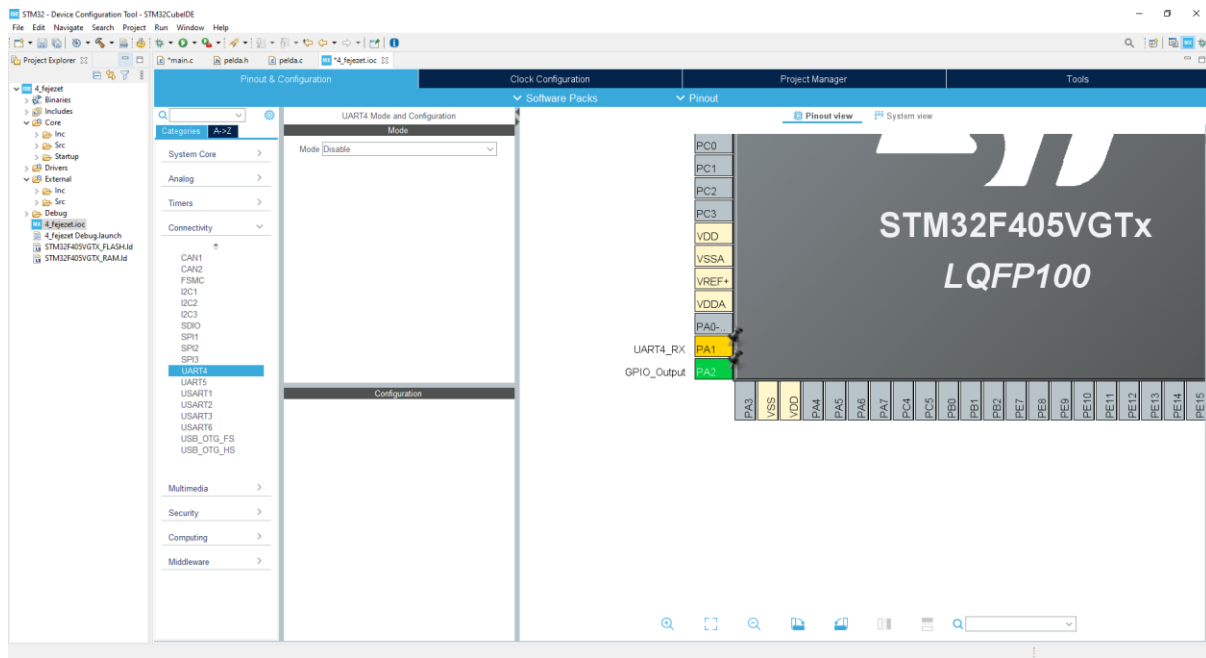
31. Ábra - Kivezetés beállítása

Előfordulhat, hogy olyan funkcionalitást választunk egy kivezetéshez, amely vizuálisan sárga jelzést eredményez a CubeMX-ben, például így:



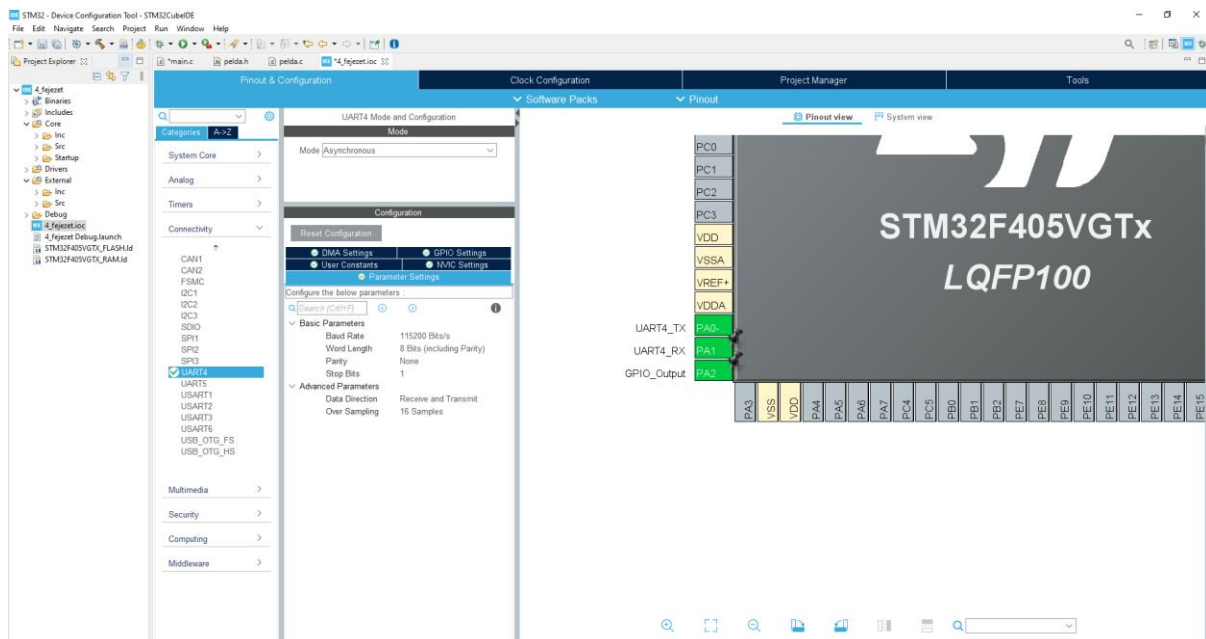
32. Ábra - Kivezetés beállítása engedélyezetlen periféria mellett

Ebben az esetben nem kell megijedni, csupán arról van szó, hogy a kivezetéshez tartozó periféria még nem lett engedélyezve és felkonfigurálva. Mivel most egy UART jelet választottunk ki, nézzünk is bele kicsit ebbe a konfigurációba:



33. Ábra - Engedélyezendő periféria kikeresése

Látható, hogy az `UART4_RX` jelhez tartozó `UART4` periféria (ami a **Connectivity** legördülő menüben található) nincs engedélyezve (**Mode** → **Disable**).

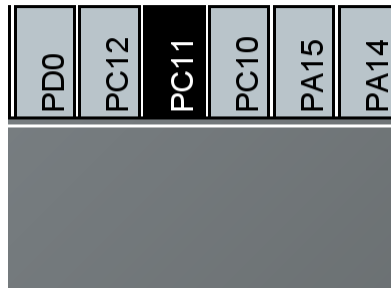


34. Ábra - Periféria engedélyezése

Ha a módot átállítjuk **Asynchronous**ra, a sárga pin egyből zöldre vált (a perifériát engedélyeztük), és megjelennek a perifériához tartozó konfigurációs beállítások. Ezeknek a konkrét beállítási paramétereiről egy későbbi tananyagrészen lesz szó.

Mit csináljunk akkor, ha mi mindenképpen szeretnénk egy `UART` kommunikációs perifériát használni, azonban az előbb kijelölt kivezetések (PA0 és PA1) már foglaltak, így oda semmiképpen sem kerülhetnek? Erre kézenfekvő megoldás, hogy végignézzük az összes kivezetést, és leteesszük oda,

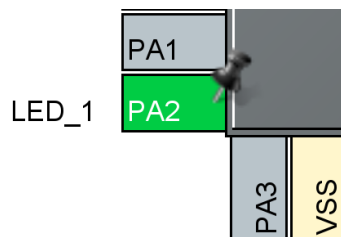
ahol van rá lehetőség. Egy 16 vagy 32 lábú mikrovezérlő esetén ez még egészen kivitelezhető, azonban egy 64, 100 vagy akár 144 kivezetéssel rendelkező esetben időigényesnek ígérkezik. A keresés meggyorsítható, ha a **Ctrl** gomb lenyomása közben rákattintunk (és nyomva is tartjuk) például az **UART4_RX** kivezetésre. A CubeMX halványkékkel megjelöli azokat a pineket, amelyek alternatívaként szolgálnak az elhelyezésre. Ez az **UART4_RX** esetében a PC11 kivezetést jelenti:



35. Ábra - Alternatív kivezetés jelölése

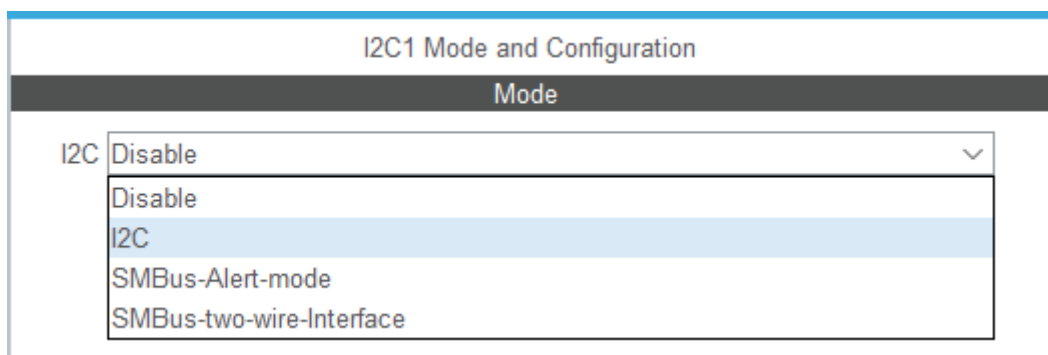
Amennyiben szeretnénk ide áthelyezni a jelet, nincs más dolgunk, mint a **Ctrl+kattintás** mellett odahúzni az egerünket, majd sötétkékre váltáskor elengedni, és az **UART4_RX** át is került az új alternatív helyére.

Ha rákattintunk a megjelölt kivezetésre a jobb egérgombbal, az "Enter user label" opció segítségével tudunk beszédesebb nevet is adni neki, ami később a programozás során jócskán leegyszerűsíti a dolgunkat. Nevezzük ezt most át **LED_1**-re.



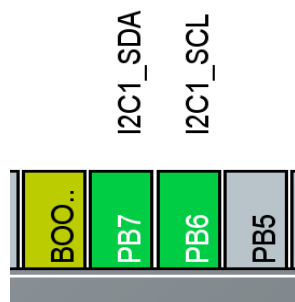
36. Ábra - Kivezetés címkézése

Az elnevezéseket, amiket adtunk, a **main.h** fájlban találhatjuk meg (generálás után), ami a projekt **Core** mappájában található **Inc** almappában van. Ezek a módosítások a bal oldalt található legördíthető elemekből is megtehetőek. Konfiguráljunk fel ezúttal egy I²C perifériát. Nyissuk le a **Connectivity** fület, és válasszuk ki az **I2C1** perifériát.



37. Ábra - I2C periféria beállítása

Állítsuk be I2C-re, majd ezt követően láthatjuk, hogy a CubeMX elhelyezte a szükséges jeleket a PB6 és PB7 kivezetésekre.



38. Ábra - Sikeresen felkonfigurált I2C-kivezetések

Természetesen ezt követően is van lehetőség az alternatív elhelyezés kiválasztására, ez csupán még egy módja a jelek kivezetéshez rendelésének.

Amennyiben végeztünk a konfigurációval, el kell mentenünk a változtatásainkat. Ezt legegyszerűbben a **Ctrl+S** billentyűkombinációval, vagy az ikonos menüsor kis floppy lemezt ábrázoló elemével tehetjük meg. A mentés során az új konfigurációhoz tartozó kódrészleteket a fejlesztőkörnyezet legenerálja nekünk, ha ezt szeretnénk, ezzel is segítve a programozást. Ha belenézünk a *main.c* állományba, láthatjuk, hogy nagyon sok zöld kommentezett sor található, melyek afféle címkéként szolgálnak a programozó számára. Fokozottan ügyeljünk, hogy a saját kódunk írása közben **CSAK** olyan helyre írjunk, amely egy nyitó és záró komment közé esik, különben a legközelebbi generáláskor a CubeMX azt kitörli. Például:

```

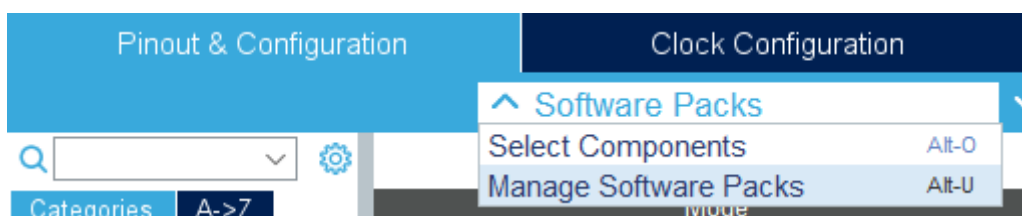
65 int main(void)
66 {
67     //EZ A SOR ELVESZIK!
68     /* USER CODE BEGIN 1 */
69     //EZ A SOR MEGMARAD
70     //EZ A SOR IS MEGMARAD
71     /* USER CODE END 1 */
72     //EZ A SOR ELVESZIK!
73     /* MCU Configuration-----*/

```

39. Ábra - Kódírási ökölszabály

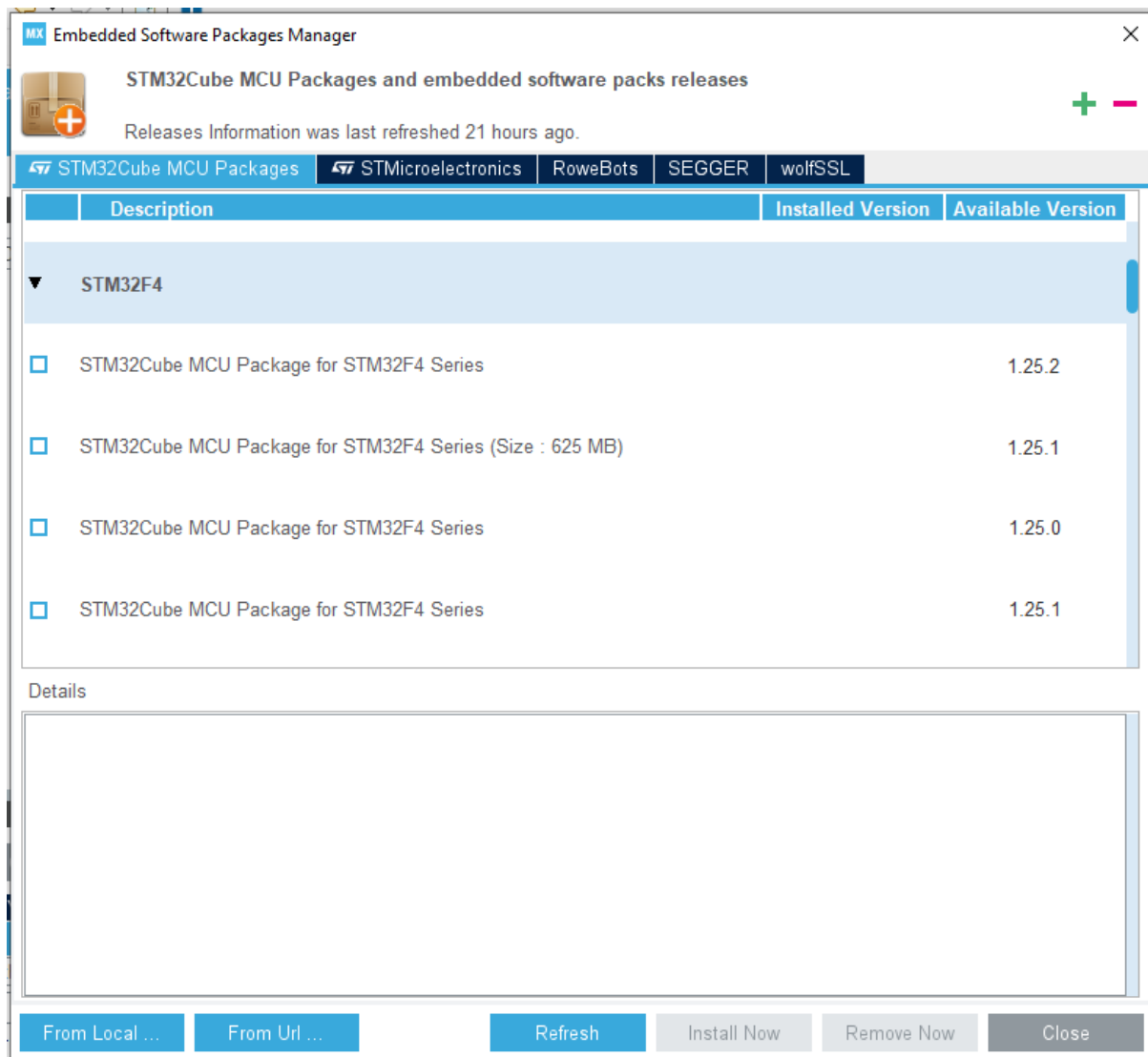
Minden esetben egy Begin és egy End tag közé írjunk csak kódot.

Amellett, hogy a CubeMX legenerálja nekünk a kódot, további lehetőséget is kínál kész kódrészletek megismerésére. Nyissuk meg a **Software Packs** legördülő menüben található **Manage Software Packs** opciót:



40. Ábra - Szoftvercsomag ablak megnyitása

A következő ablakot kell látnunk:



41. Ábra - Szoftvercsomag ablak, legfrissebb verzió telepítése

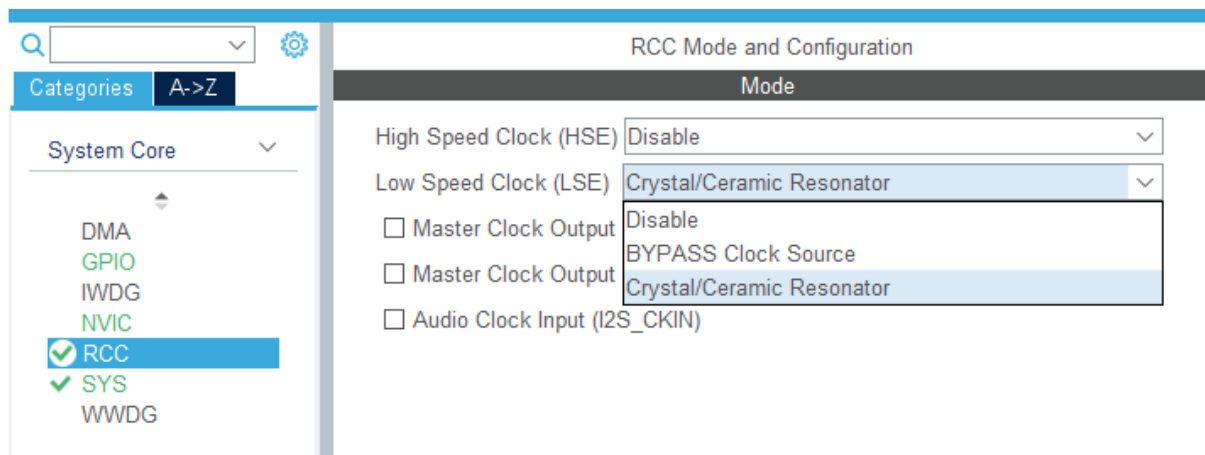
Gördítsük le a mikrokontrollerünknek megfelelő fület (esetünkben **STM32F4**). A kattintást követően el is tudjuk olvasni a **Details** fül alatt, hogy mit is tartalmaz a szoftvercsomag. Rengeteg példát, segítő függvényt rejt magában a csomag, például a *HAL* (Hardware Abstraction Layer) függvényei is ebben találhatóak meg, ezekről egy későbbi tananyagrészen lesz szó.

CLOCK CONFIGURATION

A mikrokontroller működéséhez elengedhetetlen, hogy megfelelő órajelet kapjunk. Erre több lehetőségünk is van, a konfigurátorban ezek közül választhatunk.

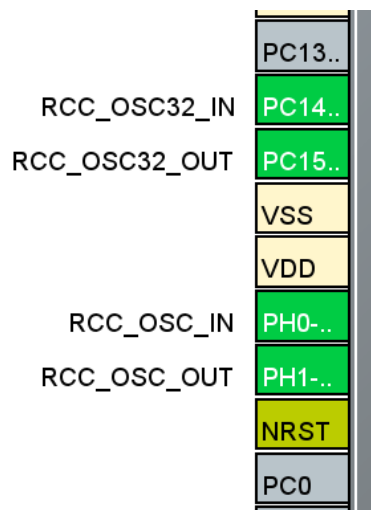
Mielőtt azonban az órajelek beállításába belevágnánk, engedélyezzük a külső oszcillátorokat, mint órajelbemeneteket.

Ehhez az előbb tárgyalt **Pinout & Configuration** fülön gördítsük le a **System Core** részt, és nyissuk meg az **RCC** (Reset and Clock Control) konfigurációját:



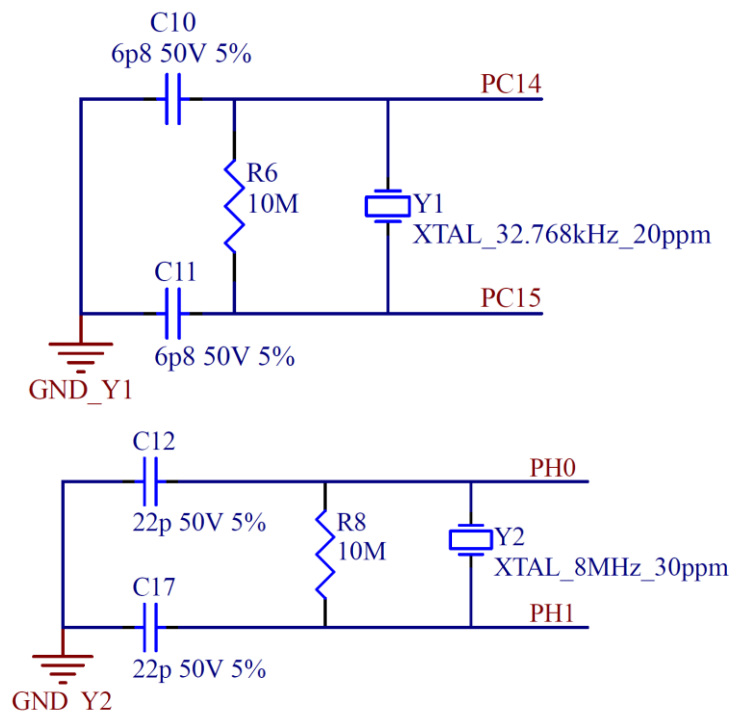
42. Ábra - Külső oscillátorok konfigurációja

Mind a **HSE** (High-Speed External), mind az **LSE** (Low-Speed External) esetében állítsuk be a *Crystal/Ceramic Resonator* opciót, mivel ez az opció jelenti a külső, kristály alapú oscillátort, amely a kártyán megvalósításra került. A beállítást követően ezek a paraméterek egyből megjelennek a megfelelő kivezetéseken is:



43. Ábra - Oscillátorok sikeres beállítása

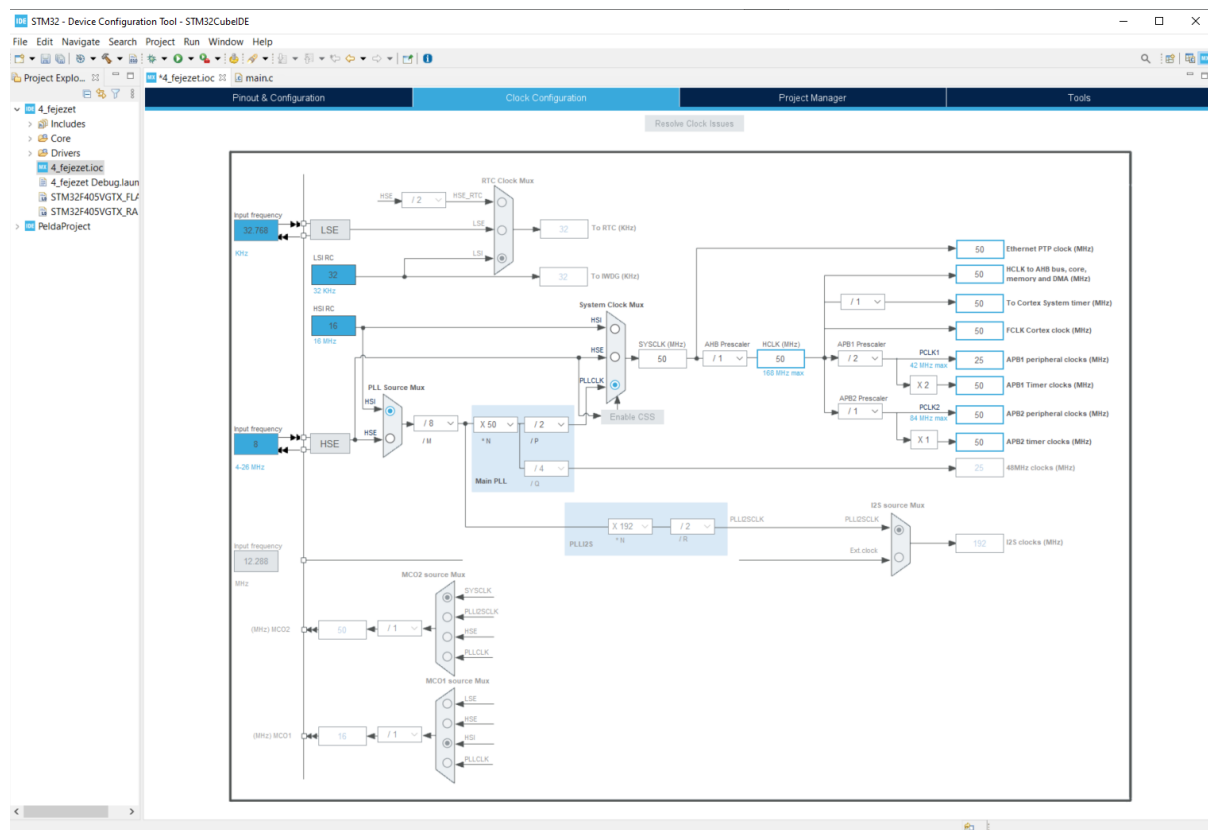
Mindenképpen győződjünk meg arról, hogy a CubeMX a hardvernek megfelelő lábakhoz rendelte hozzá az oscillátorok jeleit. Ezt legkönnyebben úgy tehetjük meg, hogy leellenőrizzük a fejlesztőkártya kapcsolási rajzában:



44. Ábra - Oszcillátorok a fejlesztőkártya kapcsolási rajzán

Láthatjuk, hogy a megfelelő PH0-PH1, illetve a PC14-PC15 lábakhoz történt a hozzárendelés. **Az ilyen beállítások elvégzése során mindig győződjünk meg arról, hogy oda konfiguráltuk be a jelet, ahova az hardveresen elő is van készítve!**

Kattintsunk át a felső **Clock Configuration** fülre. Ezen a fülön van lehetőségünk a mikrokontroller órajeleinek beállítására. A bal oldalon a rendelkezésre álló órajelforrások (a külső oszcillátorokat az előbb engedélyeztük, a belső oszcillátorok pedig az LSI és HSI RC), a jobb oldalon pedig az abból előállított órajel-frekvenciák találhatók a hozzájuk tartozó perifériák elnevezéseivel.



45. Ábra - Órajel konfigurációs panel

A kívánt órajel-frekvenciákat különböző szorzások és osztások elvégzésével állítja elő a mikrokontroller hardveres és szoftveres úton. Úgy állíthatjuk elő legegyszerűbben az általunk használni kívánt órajelet, hogy beírjuk a fenti ábrán a kékkel jelölt mezők egyikébe a szükséges frekvencia értékét, majd az **Enter** megnyomását követően a fejlesztőkörnyezet kiszámolja a szükséges beállításokat. Próbáljunk is ki egy ilyen beállítást! Állítsuk át például az **APB1 Timer Clock**-ot 80 MHz-ről 12 MHz-re. **Enter** lenyomást követően látható, hogy a különböző szorzó, osztó és multiplexer áramkörök segítségével az előzőtől egy teljesen eltérő órajel konfigurációt hoztunk létre. Megeshet, hogy olyan értéket szeretnénk kiszámoltatni, amelyet a mikrokontroller órajelhálózata nem tud előállítani, ebben az esetben ezt pirossal fogja nekünk jelölni a szoftver.

Mielőtt visszalépnénk, állítsuk vissza a régi 80 MHz-es értékre, és számoltassuk is vissza a CubeMX-el az értékeket, mivel a különböző órajelek frekvenciáinak módosítása akár nem várt változásokat is eredményezhet, így ezekkel a beállításokkal minden esetben körültekintően járunk el!

PROJECT MANAGER ÉS TOOLS

A **Project Manager** fülön adminisztratív változtatásokat eszközölhetünk (például névváltoztatás). A korábban említett szoftvercsomagok automatikus frissítését is itt tudjuk beállítani.

Mcu and Firmware Package

Mcu Reference

Firmware Package Name and Version
 ☒ Use latest available version

46. Ábra - Automatikus frissítés beállítása szoftvercsomagokhoz

Még a **Project Manager** fülön belül, az oldalsó navigálógombok segítségével menjünk át a Code Generator al-opcióra. Itt a már korábban említett automatikus kódgeneráláshoz kapcsolódó beállításokat találhatjuk meg. A fájlgenerálás résznél mindenképpen pipáljuk be az alábbi opciókat:

Generated files

☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral

☐ Backup previously generated files when re-generating

☒ Keep User Code when re-generating

☒ Delete previously generated files when not re-generated

47. Ábra - Kódgenerálási beállítások

Az első opció sokkal átláthatóbb kódot eredményez (ugyanazt csinálja, mint mi kézzel a Fájlkezelés részben). A második lehetőség biztonsági mentést készít a kódról, mielőtt újragenerálja azt. A harmadik opció felelős azért, hogy két generálás között ne vesszen el az általunk írt kód, míg az utolsó a már nem használt fájlok törléséért felel (például legeneráltattunk egy perifériához tartozó fájlt, de mégis szükség az adott perifériára, így a fejlesztőkörnyezet a következő generáláskor törli a projektből).

Lépünk át a **Tools** fülre a felső kék sorban. Itt találhatunk egy szimulációs eszközt, amelynek segítségével megnézhetjük, hogy milyen fogyasztása lesz a tervezett alkalmazásunknak (a perifériáknak). A **New Step** gomb megnyomásával tudunk kiválasztani egy szimulálandó állapotot.

Kitekintés:

CubeMX és STM32CubeIDE

Talán egyesek furcsállhatják, hogy az érződik az STM32CubeIDE és CubeMX programokon, hogy valamilyen rokonságban állnak, hiszen a logók is megegyeznek, és tulajdonképpen most úgy használjuk a CubeMX-et, mint egy programot a programban. Nos ez azért van, mert az STM32CubeIDE és a CubeMX kettő különálló program volt. Ugyanezek a funkciók voltak elérhetőek, csak kettő külön alkalmazásban. A CubeMX-ben generált projektet kellett importálni az STM32CubeIDE-be. A két program együttműködése nem volt zökkenőmentes, ezért is döntöttek úgy, hogy inkább integrálják a CubeMX-et a fejlesztőkörnyezetbe, mondván, nagy általánosságban úgyis egyszerre használják a kettőt a fejlesztők. A CubeMX ma is elérhető önállóan, hogy más fejlesztői környezettel is lehessen használni.

ST-LINK

MI IS EZ VALÓJÁBAN?

Eddig végig arról esett szó, hogy hogyan tudjuk a programunk megalkotását a fejlesztőkörnyezet segítségével minél könnyebben elvégezni. Jogosan merülhet fel a kérdés, hogy kész van a kód például egy LED villogtató programhoz, de hogy fog ez ténylegesen “rákerülni” a fejlesztőkártyára? Erre a feladatra való eszköz az STMicroelectronics mikrokontrollereihez készített “programozója”, amelyre a továbbiakban csak “ST-Link”-ként hivatkozunk.



48. Ábra - ST-Link programozó adapter

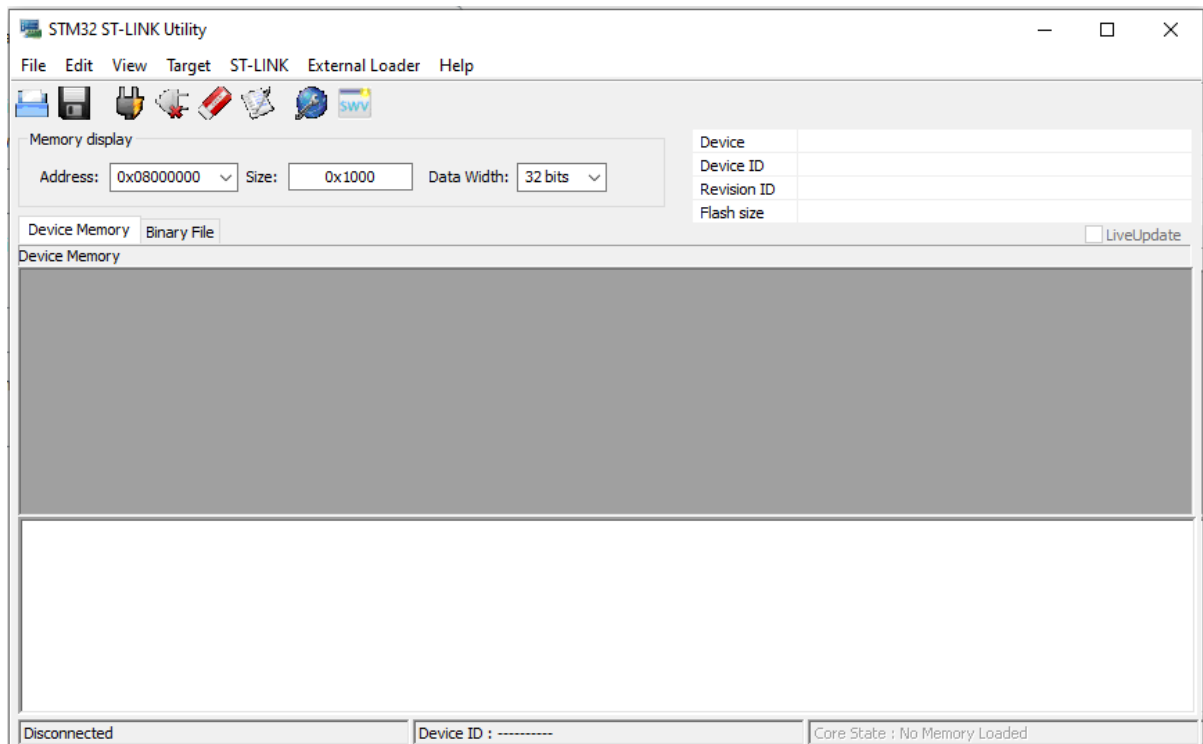
Ez tulajdonképpen maga is egy mikrokontrollert tartalmazó eszköz, amelynek feladata a program továbbítása a számítógépről a célhardver mikrokontrollerének programmemóriájába.

Ahhoz, hogy ezt megtehessek, össze kell kötnünk a két eszközt. A mikrokontroller ST-Linkkel kompatibilis interface kivezetése és a programozó közé egy 20 eres szalagkábel, a számítógép és a programozó közé pedig az USB - mini USB kábelt kell kötnünk. A kapcsolat létrejöttéről az eszköz egy státusz LED segítségével ad visszajelzést. Győződjünk meg róla, hogy a fejlesztőkörnyezetet elindítottuk, mivel, ha csak az ST-Link csatlakoztatása után indítjuk el az STM32CubeIDE-t, akkor nem fogja felismerni a programozót. Amennyiben pirosan világít, akkor valami hiba történt. Ellenőrizzük, hogy mindent jól dugtunk-e be és a fejlesztőkártya feszültség alatt van-e. Lehetséges, hogy a számítógép nem látja az ST-Linket, ezt a PC eszközkezelő alkalmazásában le tudjuk ellenőrizni az USB eszközök listájában. A sikeres kapcsolat esetén Zöld-Piros villogást kell látnunk, ami azt jelzi, hogy az eszközök kommunikálnak egymással.

Természetesen ez sem egy egyfunkciós eszköz, számos további lehetőséget kínál a programunkban történő hibakereséstől (erről egy későbbi tananyagrészen lesz szó) az eszköz memóriatartalmának megtekintéséig. Ezen funkciók egy része elérhető az STM32CubeIDE fejlesztőkörnyezetből is, azonban az ST-Link-hez szorosan kapcsolódó műveletek egy külön programban, az ST-Link Utility szoftverben is helyet kapnak.

ST-LINK UTILITY

Mielőtt megismerkednénk a szoftver által nyújtott lehetőségekkel, telepítsük fel a számítógépünkre a fejlesztőkörnyezethez hasonlóan! Ezen a linken (<https://www.st.com/en/development-tools/stsw-link004.html#get-software>) ugyanúgy a bejelentkezést követően a Get Software gombra kattintva, letölthető a program. A letöltést követően futtassuk a tömörített állományban található telepítőprogramot, majd annak befejeztével indítsuk is el a szoftvert.



49. Ábra - ST-Link Utility szoftver felülete

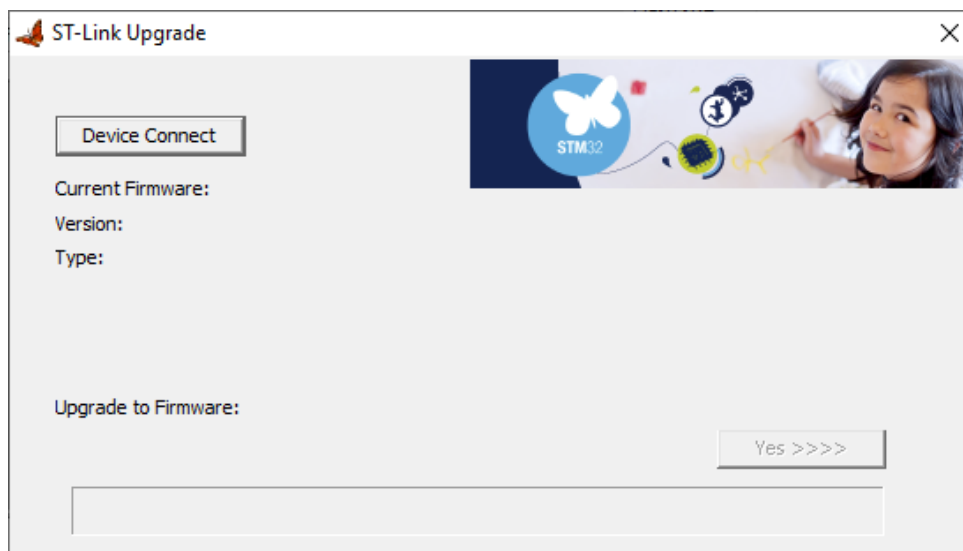
Amennyiben a programozó (ST-Link) létrehozta a fizikai kapcsolatot a számítógép és a fejlesztőkártya között, akkor a **Target** menüből a **Connect** opciót, vagy az ikon sorból a 3. (szürke-narancssárga) képet kiválasztva tudunk csatlakozni a mikrokontrollerhez.

A csatlakozást követően lehetőségünk van a mikrokontroller memóriájának módosítására: Törölhetjük teljesen az **Erase Chip** (**Target** → **Erase Chip**) gomb segítségével, vagy csak egy részét az **Erase sectors** gomb segítségével. Fel is programozhatjuk (**Target** → **Program**), amennyiben rendelkezünk a megfelelő kiterjesztésű állománnyal (.bin: bináris fájl, a buildelés kimenete). Akár össze is tudjuk hasonlítani az eszköz memóriájának tartalmát egy általunk megadott fájl tartalmával (**Target** → **Target memory compare with File**), ami esetleges programfeltöltési hibákra világíthat rá. Ezen felül az is érdekelhet bennünket, hogy egy adott szoftver mely verziója van a mikrokontrolleren, így például sokkal könnyebb meghatározni, hogy egy hiba melyik verziónál jött létre.

ELSŐ HASZNÁLAT

Mielőtt felprogramoznánk a fejlesztőkártyát az első kódunkkal, győződjünk meg róla, hogy az ST-Link-ünk a legfrissebb szoftvert használja-e (ezt az STM32CubeIDE automatikusan ellenőrzi). Amennyiben frissítenünk kell, az előbb bemutatott ST-Link Utility szoftver segítségével végezhetjük el a legkönnyebben:

A felső menüsorból válasszuk ki az **ST-LINK**, azon belül pedig a **Firmware Update** opciót. A következő felugró ablak fogad bennünket:



50. Ábra - ST-Link szoftver frissítése

Először a **Device Connect** gomb megnyomásával csatlakozunk az ST-Link-hez. Amennyiben itt hibaüzenetet kapunk, húzzuk ki, majd dugjuk be újra a számítógépbe az USB csatlakozót (ehhez a szoftverfrissítéshez nem szükséges, hogy a fejlesztőkártya is be legyen dugva az ST-Link-be).

Amennyiben a kapcsolódást követően az eszköz jelenlegi verziója eltérő a legfrissebbtől, úgy a **Yes >>>>** gombbal kezdeményezzük a frissítést.

Ha ezzel megvagyunk, jöhet egy adott program feltöltése a fejlesztőkártyára. Ehhez már mindenképpen szükséges a fejlesztőkártya csatlakoztatása is (kapcsoljuk össze a fejlesztőkártyát és az ST-Link-et a szalagkábeles csatlakozók segítségével, majd az ST-Link Utility szoftverben nyomjunk a **Connect** gombra). Ahhoz, hogy az eszközre feltöltsük a programot, meg kell találnunk a fordítás végén létrejött .bin állományt. Navigáljunk a 4. tananyagrészt projektjének mappájába. Itt lépünk be a **Debug** mappába, ahol, ha tényleg lefordítottuk a kódunkat, lennie kell egy .bin kiterjesztésű állománynak. Az ST-Link Utility szoftverben a **Target** menüből válasszuk ki a **Program...** opciót, majd keressük ki az előbb megtalált bináris állományt. A feltöltés befejeztével vessünk egy pillantást a fejlesztőkártyára, és ha mindent jól csináltunk, akkor sikeresen rátöltöttük az első beágyazott alkalmazásunkat a fejlesztői kártyára, és látnunk kell, ahogy az egyik LED villog. *(A LED villogtató program a beágyazott rendszerek Hello World-je :))*

ÖSSZEFOGLALÁS

Ebben a tananyagrészen megismerkedhettünk az STM32CubeIDE legfontosabb elemeivel, illetve azok használatával. A fejezet megismerését követően képesek lesztek önállóan használni a fejlesztőkörnyezet funkcióit, ezen felbátorodva talán egy-két saját projektet is elkezdtek majd. :)

Természetesen a tananyagrészt a fejlesztőkörnyezet csak alapvetően szükséges funkcióit ismerteti, aki szeretne még többet megtudni az STM32CubeIDE-ről, az, az ST honlapján található fejlesztőkörnyezethez tartozó dokumentációban

(https://www.st.com/resource/en/user_manual/dm00629856-stm32cubeide-user-guide-stmicroelectronics.pdf) található további információkat.

